IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Received 10 December 2025, accepted 17 January 2026, date of publication 28 January 2026, date of current version 9 February 2026.

Digital Object Identifier 10.1109/ACCESS.2026.3658339

RESEARCH ARTICLE

Experimental Evaluation of AI-Augmented Cybersecurity Requirements Generation Leveraging LLMs' Capabilities

JUAN C. YELMO, YOD-SAMUEL MARTÍN, (Member, IEEE), AND SANTIAGO PEREZ-ACUNA
Information Processing and Telecommunications Center, ETSI Telecomunicación, Universidad Politécnica de Madrid, 28040 Madrid, Spain

Corresponding author: Yod-Samuel Martín (ys.martin@upm.es)

This work was supported in part by the Spanish Ministry of Science and Innovation through the Projects Modelos y plataformas para sistemas informáticos industriales predecibles, seguros y confiables (PRESECREL) under Grant PID2021-124502OB-C43, and in part by Soluciones para la construcción de sistemas de software industriales ciberseguros e inteligentes desde el diseño basadas en inteligencia artificial que impulse la productividad y crecimiento de una economía y sociedad cibersegura (SOLSTICIA) with Funds from the Centre for Technological Development and Innovation (CDTI) and the European Union–NextGenerationEU under Grant MIG-20211006.

ABSTRACT When it comes to cybersecurity requirements, catalogs of standard controls are a double-edged sword: they offer reusable resources applicable anywhere, yet they entail tedious, error-prone manual instantiation work. As the definitions of these cybersecurity controls are written in natural language, we analyze whether large language models (LLMs) may help automate that process. We conducted an empirical evaluation where frontier versions of LLaMa, Qwen, Mixtral, and GPT were asked, leveraging best prompting practices, to generate cybersecurity requirements for a realistic system, departing from its functional specification plus definitions of abstract cybersecurity controls extracted from ISO/IEC 27002. We compared the results to a ground truth created by human experts and a gold standard pooled from the aggregation of all human- and LLM-produced results, considering phenomena such as hallucinations, new requirements discovered by LLMs, or partial overlaps. Results show that open-weights LLMs can parallel human experts in coverage and help uncover additional requirements when used under human-in-the-loop curation. Relative F2 score against the gold standard (prioritizing recall over precision) reached 1.14× in the best LLM configuration (in comparison to human experts): a LLaMa 3.1 405B model, prompted with an explicit chain-of-thought pipeline made of three sequential steps and four parallel and coalesced runs, with moderate values for hyperparameters that balance diversity and determinism. Our work contributes a benchmark of applied capabilities of LLMs for security by design in non-code artifacts. Although recall values leave room for improvement as LLMs advance, results provide initial evidence that this approach is feasible in our case study.

INDEX TERMS Computer security, large language models, requirements engineering, security management.

I. INTRODUCTION

Security-by-design treats cybersecurity as an intrinsic concern that steers the software development lifecycle from the outset rather than as a post hoc add-on. Consequently, under this paradigm, cybersecurity presents first-class requirements on a par with other quality attributes. To operationalize this principle, cybersecurity standards distill decades of practice into reusable catalogs of control definitions that organizations are expected to tailor to the context of the target system under development, early in its lifecycle. Yet, transforming those abstract, system-agnostic, cross-cutting, standard control definitions into concrete, system-specific requirements—while tracing each control through every relevant system component—remains a labor-intensive endeavor. It demands a deep understanding of both the system's application domain

The associate editor coordinating the review of this manuscript and approving it for publication was Porfirio Tramontana.

© 2026 The Authors. This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License.
For more information, see https://creativecommons.org/licenses/by-nc-nd/4.0/

and cybersecurity, and still involves painstaking, error-prone textual work. For example, the definition of one ISO/IEC 27002:2022 control recommends ''*providing physical or logical access controls for the isolation of sensitive applications, application data, or systems*''. This outlines a required capability to protect sensitive assets, but it is still an abstract statement: each project must operationalize it by instantiating the control in the context of its own target system under development (i.e., determining which applications, data, or systems are ''sensitive'', which will vary from one development project to another).

Recent advances in large language models (LLMs) create an opportunity to reduce this burden by assisting requirements engineers in generating system-specific cybersecurity requirements derived from standardized cybersecurity control definitions, which many standards express as parameterized templates. Instruction-following LLMs have demonstrated an emergent ability to obey complex prompts, reason over linguistic patterns, and combine information from multiple sources—skills well suited to automating the instantiation of standard control definitions into system-specific requirements. Nevertheless, evidence for their practical efficacy in this task remains scarce: empirical software-engineering studies on LLM applications tend to focus on code-centric tasks, whereas those in cybersecurity examine systems already in operation rather than their requirements.

This work investigates whether state-of-the-art, widely available LLMs—with a special emphasis on frontier, open-weights models—can assist requirements engineers who lack deep cybersecurity expertise in deriving system-specific cybersecurity requirements from ISO/IEC 27002:2022. We address this question through an in-depth, expert-validated empirical evaluation in which four LLMs from leading providers—Meta, Alibaba, Mistral, and OpenAI—are benchmarked on a curated subset of ISO/IEC 27002 control definitions for a constructed yet realistic specification of a smart automotive manufacturing system (AI4I4 – Automated Identification for Industry 4.0). This specification serves as our testbed and provides the functional and organizational complexity needed to exercise a broad range of security controls. Each model is prompted to instantiate the selected controls for this system, and we then compare their output against a benchmark manually produced by a panel of human experts, under a preregistered rubric that rewards coverage while penalizing hallucinations. By focusing on a single representative system and a carefully selected subset of controls, we deliberately trade breadth for depth, making it feasible to construct an expert-validated ground truth and carry out the intensive human evaluation needed to assess LLM behavior in a realistic security-by-design setting.

Our contributions are threefold: 1) a systematic methodological framework and testbed to benchmark the applied competence of LLMs in generating system-specific, cybersecurity requirements derived from ISO/IEC 27002 control definitions; 2) a concrete instantiation of this framework, in the form of a publicly released prompting-and-instantiation pipeline and evaluation dataset, which facilitate replication and extension and which we use to constructively demonstrate what current LLMs can achieve on our task under best-effort prompting; and 3) an empirical characterization of performance across models, prompts, hyperparameters, and run-to-run variability.

The remainder of the article is organized as follows. Section II introduces the research questions, which examine the LLM applied competence in our task and the effects of different variables. Section III lays the conceptual foundation: presents a concise taxonomy of LLMs capabilities, surveys prior work on LLM adoption in requirements engineering and defensive cybersecurity, reviews methods for deriving cybersecurity requirements from control standards, and synthesizes evaluation techniques for assessing LLMs in cybersecurity tasks. Section IV describes our method, including experimental evaluation design, input and output artifacts, prompting strategies, and evaluation metrics. Section V presents quantitative and qualitative results, while Section VI analyzes those findings in light of our research questions, outlines practical implications, and examines threats to validity. Finally, Section VII concludes with avenues for future research. In addition, we have released research artifacts to support reproducibility.

II. RESEARCH QUESTIONS

Our work explores the feasibility and practical limits of using LLMs to automate the generation of system-specific cybersecurity requirements derived from the instantiation of abstract, system-agnostic definitions of cybersecurity controls found in international standards. Building on the motivation outlined above in Section I, we pose one primary research question on overall effectiveness (RQ1), and a second research question on how this effectiveness depends on key aspects of model selection and use, decomposed into four sub-questions (RQ2.1–RQ2.4) that follow a one-factor-at-a-time experimental design.

- RQ 1. Overall effectiveness: *To what extent can current LLMs automatically instantiate standard cybersecurity control definitions into system-specific cybersecurity requirements, measured by coverage and correctness, compared with requirements generated manually by experts under the same task?*
  This question establishes a baseline for the task: given careful tuning of models, we measure how far LLMs can go in matching experts in terms of coverage and correctness.

After establishing this baseline, we address how individual configuration choices influence performance. We consider four dimensions—model choice, prompt design, decoding hyperparameters, and run-to-run stochastic variability—and, for each, vary one factor at a time while keeping the others fixed. This leads to the following four sub-questions:

- RQ 2.1. Model choice: *How does the quality of generated cybersecurity requirements vary across different frontier LLMs?*

This question isolates the impact of the underlying model family. We pay particular attention to open-weights models that can be deployed on-premises and audited, because many organizations must balance performance with confidentiality, deployment flexibility, and compliance constraints.

- RQ 2.2. Prompt design: *How does prompt design (e.g., wording, structure, chaining) affect the quality of generated cybersecurity requirements?*
  Since LLM behavior is highly prompt-dependent, this question examines the sensitivity of task performance to different prompt designs. We construct prompts of varying strength, including intentionally underspecified versions, and iteratively refine them to draw out the strongest capabilities of each model and approximate their optimum performance for the task.
- RQ 2.3. Hyperparameters: *How do LLM decoding hyperparameters affect the quality of generated cybersecurity requirements?*
  This question focuses on common decoding settings, such as temperature and top-p, and assesses how much they influence the quality of generated requirements when all other factors are controlled.
- RQ 2.4. Run-to-run variability: *To what extent does output variability across successive runs with identical settings affect the reliability and reproducibility of using LLMs for this task?*
  Because LLMs rely on stochastic sampling and are inherently nondeterministic, we explicitly quantify variation across repeated runs with identical settings, compare it to variation between different models, and discuss its implications for assurance and reproducibility.

III. RELATED WORK

This section situates our contribution within the broader landscape of large language models and cybersecurity requirements engineering. We first clarify key terminology and categorization of LLMs—covering architectural families, operation modes, and configuration approaches (Section III-A)—to lay the groundwork that informs our choices for model selection, prompting strategies, and parameter settings (later explained within Section IV). We then survey how LLMs have been applied to requirements engineering (Section III-B) and defensive cybersecurity tasks (Section III-C), emphasizing the currently limited focus on cybersecurity requirements generation and demonstrating how our approach advances beyond prior work. To complement this, Table 9 in Appendix A provides a summarized comparison of applications of LLMs in related areas. Next, we review the evaluation methodologies for LLMs (Section III-D), highlighting the shortcomings of purely automated reference-based metrics when assessing practical procedural knowledge—which motivates our adoption of human-driven rubrics. Finally, we outline the established practice for deriving system-specific requirements from abstract control definitions coming from cybersecurity standards (Section III-E); which illustrates the process we address to evaluate the effectiveness of LLMs in that role.

A. OVERVIEW OF LARGE LANGUAGE MODELS: FEATURES AND CATEGORIZATION

Large language models (LLMs) are advanced neural network systems trained to reproduce linguistic patterns learned from extensive text corpora whose scale (in data size, neural parameters, and architectural complexity) makes them exhibit broad, general-purpose language capabilities. These LLMs have gained rapid prominence since the release of interactive, turn-based, chat-like user interfaces (e.g., ChatGPT in November 2022 [1]) that have popularized flexible, natural language interactions which align with user intent and are effectively redefining human-computer interaction [2].

Modern LLMs generally adopt the Transformer architecture, which uses 'attention' mechanisms to capture the textual context within their input and the previously generated output. Transformer-based LLMs can be grouped into three categories [3]: 1) *encoder-only* models focus on creating a vector representation of input text that captures its meaning for, e.g., classification or labeling tasks; 2) *decoder-only* models autoregressively generate a sequence of text tokens, which they predict taking prior context into account, for tasks such as creative writing or code generation; and 3) *encoder-decoder* models transform one text into another for, e.g., translation or summarization. Regarding decoder-only models, these may come in two flavors: *foundation models* [4] have been trained to predict the next token in a text and show general-purpose, task-independent language capabilities; while *instruction-following models* [5] are fine-tuned to interpret and execute user instructions, allowing them to produce coherent, human-like responses across diverse tasks, effectively answering questions, generating structured responses, and following multi-step prompts.

While LLM pretraining allows them to learn general language representations from massive text datasets, they may then undergo *fine-tuning* on smaller task-specific datasets to optimize performance for specialized tasks. However, the sheer size of the training process also allows LLMs to exhibit emergent capabilities—skills such as reasoning or summarization for which they were not explicitly trained [6], [7], [8]—, without any further fine-tuning. Likewise, more specialized behaviors can be elicited from instruction-following LLMs through *prompt-based learning*, that is, prompting the LLM with carefully crafted inputs without retraining the model parameters. For instance, few-shot learning allows an LLM to infer how to handle a new task by being prompted with examples of paired inputs and outputs for that task, while zero-shot learning relies solely on LLM capabilities to apply reasoning patterns to the specific instructions provided as input [9], [10], [11], [12], [13]. This adaptability makes them suitable for a wide array of open-ended applications and has contributed to their popularization. Notwithstanding such capabilities, LLMs also display potential drawbacks,

as they can generate plausible but incorrect responses (*hallucinations*), propagate biases from training data, and demand intensive computational resources [14].

Decoder-only LLMs typically employ probabilistic sampling; thus, repeated runs with the same prompt may yield different results. While this behavior can improve creativity and reduce repetitive loops in the output, it may come at the cost of reduced consistency and coherence. Hence, LLMs let their users manage such behavior through the configuration of *hyperparameters*: temperature (which governs the randomness in selecting the next output token among candidates), top-p and top-k (which restrict sampling to the most probable tokens), and additional penalties that further encourage diversity in generated responses (e.g., frequency or presence penalty). The configuration of these hyperparameters allows the user to choose among a range of behaviors from the LLM, between more creative, diverse responses on one end, and coherent and deterministic on the other end. Empirical studies confirm that these decoding choices materially affect output diversity, coherence, and factual reliability: higher temperatures and broader nuclei increase novelty at the cost of a higher risk of incoherence and factual errors, whereas more conservative settings tend to yield safer but more repetitive text, with consequences that vary across tasks [15], [16]. This sensitivity is also reflected in the documentation issued by LLM providers [17], which explicitly recommends tuning hyperparameters per use case rather than relying on a single default configuration.

From the standpoint of deployment and exploitation, LLMs also vary in how (and if) their code, training data, or weights are shared. Whereas many of the models that have gained public recognition are subject to a *proprietary* and restrictive access framework and can only be used through provider-operated online service interfaces, open-weights models [18], in contrast, release their trained weights under permissive licenses, which enable local deployment and customization. Local execution can support regulatory compliance, strong confidentiality guarantees and fine-grained auditing, but it typically shifts costs from per-call API (application programming interface) charges to higher up-front capital expenditure on hardware, with lower marginal operating costs (mainly electricity). Empirical analyses of cost further indicate that open-weights models often require more tokens or computation than leading closed models to reach comparable performance [19], so a lower price per token does not automatically imply a lower cost per task. All in all, a trade-off exists in which self-hosted local deployments are less economical at low utilization but may become competitive against per-API services once usage exceeds certain thresholds [20], [21]. A third option is to run open-weights models as managed cloud services, which shift capital costs to operating costs related to self-hosting, and offer an intermediate position in terms of confidentiality and auditability when deployed on platforms an organization can instrument and trust under existing contractual agreements.

A final and relevant concept is that of *frontier* models: those at the cutting edge of LLM capabilities, surpassing the prior state-of-the-art within the previous year [22].

B. APPLICATIONS OF LLMS TO REQUIREMENTS ENGINEERING

Software engineering (SE) activities often involve recurring text analysis and generation tasks, which makes them suitable for the application of LLMs. Researchers have thus envisioned that LLMs can supplement, automate, or even reshape SE tasks, sharing roles with human engineers to reduce cognitive load and improve efficiency. Yet, despite a variety of vision papers [23], research agendas [24], editorials [25], and surveys [26] advocating LLM adoption across the full spectrum of software engineering, empirical research remains heavily skewed toward code-generation with specialized code-oriented LLMs. Consequently, practical experience with the deployment of general-purpose LLMs in other SE domains remains largely uncharted territory—particularly for models other than ChatGPT.

Requirements Engineering (RE) has been identified as one of the software engineering (SE) disciplines where the greatest impact is expected from the use of LLMs, due to its natural focus on text [27] and the coverage they can achieve [25]. However, their applications are still limited, typically addressing closed-ended and classification matters. Recent reviews of LLMs for requirements engineering highlight a complementary aspect of the current evidence base [28], [29]: most published studies are still lab-based exploratory evaluations on isolated activities, with relatively few field studies or deeply integrated industrial workflows.

Early studies approached requirement elicitation by mining existing artifacts [30], [31], [32], [33], [34], [35]: prose descriptions, stakeholder inputs, or even reverse-engineered source code—rather than drafting full-fledged new requirements. Indeed, current LLM applications in RE function primarily as information-extraction tools: encoder models are fine-tuned or retrained to surface and label information embedded in requirements artifacts [36], [37], [38], [39], [40], [41] and pull candidate requirements from heterogeneous repositories [42], [43]; whereas decoder models rely on in-prompt learning to address requirements classification tasks in a broad sense—category assignment [44], [45], conflict detection [46], [47], or quality appraisal [48], [49]—.

Recent practical studies illustrate both the promises and the current limitations of LLM-based requirements elicitation. [50] showed that a tightly structured interview with ChatGPT can recover part of an expert's requirement set while introducing additional (occasionally erroneous) items. Reference [44] expanded a requirements specification using GPT-3.5, with 60% of the new requirements rated both relevant and novel. Reference [49] reported that a two-pass drafting of a requirements specification by GPT-4 and Code Llama departing from a generic system description achieved comparable quality, completeness and consistency to those

created by entry-level engineers, and [51] confirmed ChatGPT's usefulness as an assistant to juniors for requirements generation. The stochastic behavior of LLMs also pose concerns regarding reliability, correctness, and interpretability of the artifacts they generate [52], which, combined with a façade of trustworthiness may end up in mistakes whose fixing may require human intervention that erodes the benefits achieved from automation [53].

In what may be closer to our work, Ronanki et al. [54] found that a catalog of trustworthiness requirements compiled by ChatGPT-3.5 outperformed humans on abstraction, atomicity, and understandability; however, they did not consider the context of the target system under development, hence their output remains a set of project-independent templates that still require system-specific instantiation—the very point where our work departs from. Likewise, Bowen [55] applied a chain of two prompts to, first, decide whether a standard applies to a product description, and second, derive concrete system requirements from the standard—yet working on a standard as a whole, rather than on a per-clause basis.

C. LARGE LANGUAGE MODELS FOR DEFENSIVE CYBERSECURITY TASKS

The application of LLMs to defensive cybersecurity tasks has primarily focused on activities that involve systems already coded or even operational: code generation and repair, static code analysis, fuzz testing, vulnerability and malware intelligence, incident reporting, or forensic analysis [7], [23], [56], [57], [58], [59], [60]. Even within the applications of LLMs to cybersecurity-related natural language documents, their scope typically remains limited to post-development artifacts, e.g., threat or vulnerability reports [61], [62]

However, broader roles have been proposed [23], suggesting that LLMs might significantly support earlier activities such as refining system specifications, filling in missing cybersecurity details, and aiding in domain modeling by identifying core entities in system descriptions. Emphasis has also been placed on leveraging LLMs to replace or complement human efforts in critical cybersecurity tasks, serving as ''cybersecurity copilots'' that manage complex data in enterprise contexts [7]. Our research specifically addresses these gaps, highlighting the potential of applying LLMs to developing cybersecurity requirements—an area currently underrepresented but ideally suited due to its linguistic complexity, analytical nature, and alignment with the security-by-design approach.

We also indirectly tackle other key issues and common challenges that have been identified regarding the use of LLMs in cybersecurity. Many are related to the associated computational resource demands and throughput limitations: the compute power required by model customization through fine-tuning makes it more advisable to rely on prompting-based configuration techniques supported by hyperparameter adjustment, while scalability, efficiency, and limitations in the size of inputs and outputs call for a strategy that divides the tasks into multiple requests to the LLM. Other significant concerns are the confidentiality of LLMs as a service, and engineers' reluctance to delegate high-level responsibilities [63] that make a case for the use of open-weights LLMs that can be deployed under user control; the freshness of testing data for evaluation not previously leaked into the LLM training dataset; and addressing hallucinations—plausible but incorrect outputs, even sometimes supported by persuasive post-hoc explanations—; as well as instability in outputs, which motivates us to examine run-to-run variability.

D. APPROACHES FOR EVALUATION OF LLMS: THEORETICAL VERSUS APPLIED KNOWLEDGE

As our RQs focus on evaluating the performance and effectiveness of LLMs in addressing a specific task—namely, generating cybersecurity requirements derived from standard control definitions—, we are interested in the existing approaches to assess their capabilities. Such functional assessment of LLMs [64], [65], [66] typically covers general-purpose tasks involving natural language understanding and processing, embedded world knowledge, and reasoning abilities; but they may also extend to address purpose-driven, domain-specific applications relevant to particular fields. Common evaluation methods involve comparing model outputs to predetermined ground truths, and applying classification metrics (e.g., accuracy, F1 score, Matthews Correlation Coefficient, Cohen's Kappa) [66] for closed-ended tasks, and textual similarity metrics (such as BLEU, ROUGE, METEOR) for generative tasks, e.g., in the scope of software engineering [26], [67].

The evaluation procedures underlying these metrics are amenable to automation, which has made them preferred for massive, repeatable evaluations of LLM performance. However, most of these only address the conceptual understanding exhibited by LLMs (''*knowledge-what*'') by assessing the conformity of the output with respect to a reference answer, while they are inadequate for assessing practical or procedural knowledge (''*knowledge-how*''). Incorporating human judgment thus becomes vital in evaluating accuracy, relevance, or fluency [64], all of which rely on nuanced semantics that are crucial for complex, open-ended tasks like ours. This qualitative approach is particularly valuable in software engineering contexts, where evaluating factuality, coherence, and depth presents significant challenges [23] Indeed, previous work cited above on the applications of LLMs to requirements elicitation [44], [49], [50], [55], [68] has addressed the evaluation of the results provided by LLMs by judging the quality features of the individual requirements generated (e.g., abstraction, atomicity, correctness, relevance, understandability, unambiguity, or verifiability) and of the overall requirements specification as a whole (consistency, nonredundancy, completeness, and conciseness).

However, the benchmarks that evaluate LLMs performance in cybersecurity tasks usually consist of multiple-choice or closed-ended questions [23], [56], [69], [70]—although,

as just stated, these might not be appropriate to evaluate the practical application of knowledge. In this sense, Carnegie Mellon University Software Engineering Institute (SEI) has indeed highlighted [71] that the evaluation of LLM applications for cybersecurity should employ the same measuring rod as what is used for human cybersecurity skills, not only relying on the accuracy of factual knowledge but on their application in the context of real-world scenarios. This evaluation of practical and applied knowledge faces three hurdles that require human involvement: crafting questions, administering them, and applying a rubric to judge the quality of the response. We will follow those guidelines when addressing our method (Section IV).

Aside from all those considerations, LLM evaluation methods play another complementary role in our research, since the identification of frontier (i.e., state-of-the-art) LLMs is integral to determining the scope of our research. To achieve this, we may resort to published results from arena-style evaluations [72], [73], where volunteers blindly compare outputs from various models responding to identical prompts, and the preferences they expressed are aggregated into rankings that reflect overall model effectiveness. Alternatively, advanced LLMs are occasionally employed as evaluators themselves, facilitating more automated and refined judgments [74], [75]. Aggregate leaderboards consolidate the results of evaluations that use these and other methods, offering comprehensive performance comparisons across multiple dimensions of evaluation [72], [73], [76], [77].

E. STANDARDS-BASED CYBERSECURITY REQUIREMENTS ENGINEERING

After reviewing related applications of LLMs, we now shift our focus to the other relevant field for our research, to introduce how the standards-based cybersecurity requirements generation process we deal with is commonly applied in practice.

The security-by-design approach fosters that analysts and requirements engineers introduce cybersecurity requirements from the outset of a project, on equal footing with other functional and non-functional requirements (rather than analyzing cybersecurity issues of an already built system, or even one in operation). Quite often, these cybersecurity requirements are stated in a 'negative' way, following risk management and threat analysis tasks that help to deduce behaviors to be prevented to avoid undesirable effects. A complementary approach consists in defining cybersecurity requirements in a 'positive' fashion from a goal-oriented perspective, i.e., as features to be implemented by the system to meet some stakeholder's goals. In this respect, cybersecurity extensions have long been developed for major Goal-Oriented Requirements Engineering (GORE) frameworks [78], [79], [80] to capture stakeholders' cybersecurity goals and refine them into operational requirements.

In order to generate cybersecurity requirements through the refinement of abstract goals, requirements can be derived from project-independent standards, which leverage pre-existing resources specific to the field of cybersecurity but independent of any particular target system under development. This approach seeks to facilitate the integration of cybersecurity into requirements engineering within the development process, as these resources encapsulate the wisdom of the community of practice in cybersecurity and can be reused across multiple development projects—eliminating the need for all engineers involved to be cybersecurity experts.

These standards usually define a series of cybersecurity controls that prescribe capabilities, constraints, or behaviors that a system must exhibit to manage specified cybersecurity risks and ensure cybersecurity properties that protect their assets. The control definitions provided by cybersecurity standards are provided at a high level of abstraction, merely stating the security issue addressed by the control, its scope, rationale, and application guidance in broad strokes. Thus, standard control definitions are merely a starting point for the requirements engineering process, as their effective application requires operationalizing them to meet the unique domain of a given target system under development.

For instance, both ISO/IEC 15408 (''Common Criteria'') [81] and NIST SP 800-53 [82] (supplemented by SP 800-53B [83] and SP 800-37 [84]) specify a series of such high-level, system-independent cybersecurity and privacy control definitions, written as parameterized templates to be instantiated by populating these parameters when the standard is used during the development of a system. The templates feature placeholders enclosed in brackets, intended as parameters that can be replaced by specific, organization-defined values relevant to the context of the target system under development, thus facilitating their customized application. Besides, both standards also define companion structured methodologies for this process to systematically convert generic cybersecurity control definitions into system-specific, precisely instantiated requirements—an approach that significantly enhances flexibility and effectiveness by closely aligning cybersecurity measures to the unique characteristics of each system.

For this purpose, each of these two standards defines a similar series of steps that an organization would take when deriving cybersecurity requirements from standard control definitions. Whether departing from a standard-defined baseline or a bespoke control selection process, they need to go through a tailoring process that involves:

1) Identifying applicable controls.
2) Applying scoping considerations.
3) Choosing compensating controls if necessary.
4) Assigning parameter values, either selecting them from among a set of constrained choices or assigning them freely defined values.
5) Iterating the same control over multiple system elements, with varying parameter values.
6) Supplementing with additional bespoke controls.

7) Detailing control implementation specifications.

8) Refining the application of controls, narrowing their applicability, or providing additional detail based on contextual specifics.

In contrast, other standards, such as ISO/IEC 27002, are not structured in that way—yet they still entail a selection, adaptation, and instantiation process, which motivates us to tackle this standard, as it might benefit more from an automation process.

In all cases, these resources are inherently extensive (as they must address a wide range of scenarios) and abstract (as they are not tailored to the specific details of any one system), and thus fall short of fully enabling and encouraging the seamless integration of cybersecurity throughout the system development lifecycle (SDLC) or the adoption of a security-by-design approach—hence our call for automation of this process to foster its adoption. Our work focuses specifically on the use of LLMs to automate steps 2, 4, 5, and 8 of the above list.

IV. EXPERIMENTAL EVALUATION METHOD

We have designed and run a structured evaluation experiment with LLMs to probe their practical competence in cybersecurity when asked to act as requirements engineering assistants in the task of deriving system-specific cybersecurity requirements from ISO/IEC 27002 control definitions, given the description of a target system under development as their input. From a Technology Readiness Level (TRL) perspective, this study corresponds to an early stage, controlled, laboratory-style benchmarking of the out-of-the-box capabilities of several LLMs on a single, well-specified case, rather than an evaluation of a deployed toolchain in industrial practice (consistently with the current maturity of the LLM-for-RE research summarized in Section III-B).

Each candidate model receives the same prompt for every chosen standard control definition, performs the sequence of generation steps described below without human interaction, and returns its answer in a structured format. The same steps were carried out beforehand by the authors in their role of cybersecurity-savvy requirements engineers. The two result sets are then compared according to a preregistered rubric, which we address as an information retrieval problem from an open set, where recall and precision metrics can be computed under reasonable assumptions.

A. OVERALL METHOD GUIDELINES

Our method responds to the 14 recommendations of the Software Engineering Institute (SEI) for evaluating LLMs in cybersecurity [71], which we introduce in this subsection. Central to these is the call for evaluating *practical knowledge* rather than mere *textbook understanding*. Accordingly, we focus on the ability of LLMs to apply cybersecurity competence to realistic tasks—rather than on their capacity to recite standard definitions or answer multiple-choice quizzes—even though this shall involve substantial manual judgment to assess the quality of the results. Consequently, we deliberately prioritize depth over breadth: focusing on a single, well-understood case enables the intensive human evaluation required for our analysis, at the expense of a necessarily limited coverage of systems, standards, and controls.

We have applied these recommendations in the creation, administration, collection, and evaluation of our experiment.

First (Section IV-B), we clearly delimit the task we are evaluating (abiding to the recommendations to *"Define an appropriate scope", "Clarify whether you are evaluating risk or evaluating capabilities"*) as the goal-oriented generation of requirements specific to a target system under development, derived from a curated subset of ISO/IEC 27002 control definitions. To mirror practitioner workflows (*"Consider how humans do it"*), both human experts and LLMs shall perform the identical three-step requirements generation process (Section IV-C), namely: 1) labeling whether each candidate standard control definition is relevant or out of scope for the target system, 2) mapping placeholders from a parameterized control template (e.g., "the user") into entities found in the system domain context (e.g., "the plant manager"); and 3) instantiating the requirement so that those placeholders are replaced with the terms mapped in clear, system-aware language. Humans and LLMs alike only receive as input the artifacts they would realistically have at their disposal—the text of the parameterized control templates and the specification of a target system under development purposely crafted for our evaluation experiment—without any extra hints ("*Incorporate affordances to humans where appropriate*" and "*Avoid affordances to humans where inappropriate*"). By pairing a never-before-released original system specification with the public ISO/IEC standard, we guard against previous leakage of our input within the LLM training data ("*Use caution with existing datasets*" and "*Beware of training data contamination*").

When selecting and configuring models (Section IV-D), we evaluate four frontier LLMs from diverse sources under conditions as similar as possible ("*Be careful with model selection bias*").

To implement SEI's guidance to "*Develop an infrastructure to support the evaluation*", a LangChain-based testbed (Section IV-F) has been developed to access models deployed across cloud-based services—from model-independent services such as Amazon Bedrock and Hugging Face Inference Endpoints, to others tailored to models by Mistral (La Plateforme) and OpenAI (GPT). This setup automates prompt dispatch, output capture, prompt-chaining orchestration, and exhaustive metadata logging. The specific prompt techniques that we have leveraged are described in Section IV-E.

Regarding the data analysis method (Section IV-G), mapping between human- and LLM-produced requirements is scored with a preregistered rubric that categorizes the correspondence between requirements (as *match*, *partial match*, or *no match*, also allowing for many-to-many mappings), enabling precision, recall, and F-measure calculations—and responding to SEI's recommendation to "*Use preregistration*". Because the intended deployment would work as an

assistant to humans rather than as a fully autonomous engineer ("*Define tasks based on intended use*"), the rubric awards credit for meaningful overlap with the human baseline, and our evaluation prioritizes coverage over absolute correctness (relying on a human analyst to reject false positives afterward). The best- and worst-case performance bounds are computed from the different prompt approaches evaluated, computations of partial matches, and estimates of confidence intervals ("*Estimate best-case and worst-case performance*"). Robustness is explored through repeated runs: on the one hand, using a single prompt template in repeated executions, and on the other hand, trying zero-shot, few-shot, and automatic chain-of-thought prompt approaches, so as to discard opportunistic tuning ("*Apply realistic perturbations to inputs*").

Last, as discussed in Section VI below, we limit any claims strictly to our scenario of standards-derived cybersecurity requirements generation ("*Avoid overgeneralized claims*"), leaving broader questions for future work.

B. SPECIFICATION OF THE TASK UNDER EVALUATION

As motivated by the RQs above presented in Section II, our task of interest is the generation of cybersecurity requirements for a target system under development; addressed through the process of operationalizing abstract, generic definitions of cybersecurity controls coming from standards (expressed as parameterized control templates), by instantiating them into system-specific and concrete requirements that should be met by the target system under development.

The requirements are thus derived by instantiating the relevant cybersecurity controls from the chosen standards, for the textual specification of the target system (Figure 1). The task is defined identically for human engineers and automated solutions alike: for each control, both receive the parameterized template together with the description of the target system under development and must produce system-specific requirements by filling in the placeholders; no additional hand-crafted hints are provided beyond the annotations already encoded in the templates. Table 1 illustrates this process end-to-end: starting from the original ISO/IEC 27002 control definition, we derive an annotated template with placeholders, which is then instantiated for a target system (AI4I4) by substituting system-specific entities into those placeholders. The remainder of this subsection details each input and output.

1) CYBERSECURITY CONTROL TEMPLATES

A standard control definition prescribes a capability, constraint, or behavior that a system must exhibit to manage specified cybersecurity risks and ensure cybersecurity properties. Standards write these control definitions at a deliberately high level of abstraction—describing the security issue, its rationale, and broad application guidance—so that they remain system-agnostic.

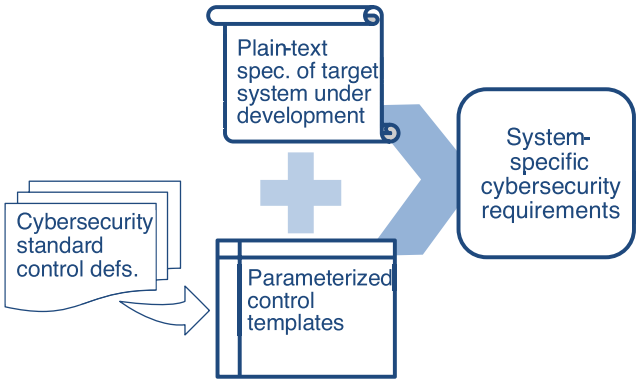


FIGURE 1. Standards-driven cybersecurity requirements generation task.

The requirements engineering process begins with these standard control definitions; each of which must be later instantiated within the context of a target system under development. When standards already offer parameterized control templates—e.g., NIST SP 800-53, Common Criteria—, these artifacts can be used verbatim. In our case, in contrast, ISO/IEC 27002 only provides high-level, unstructured, control definitions, hence we manually preprocess them and create a parameterized control template from each. The template keeps the original semantics but: 1) recasts the prose as an imperative statement, and 2) annotates generic terms and phrases (such as organization, users, roles, data, systems, locations and processes) that need replacing in the domain-specific context as brace-delimited placeholders (see Table 1 below). Although this preprocessing step requires upfront manual effort, each template is produced only once per control and remains system-agnostic, so it can be reused unchanged across future projects without repeating the annotation step.

Because ISO/IEC 27002 controls are defined as plain, non-templated prose and therefore require an additional templating step before they can be instantiated for a specific system, we regard it as a more demanding testbed than other standards that already present controls as parameterized templates organized in catalogs with explicit instantiation guidance. Focusing on ISO/IEC 27002 therefore poses a more challenging use case and allows us to study whether LLMs can bridge the gap from comparatively unstructured standard text to system-specific cybersecurity requirements. Other standards could likewise be used as starting points to derive cybersecurity requirements in different domains and contexts.

For our evaluation study, we address those controls defined, respectively, in subclauses 8.2, 8.3, 8.5, 8.20, and 8.24 of ISO/IEC 27002:2022, spanning several categories of cybersecurity practices related to secure access control and protection of data at rest and in transit. From these subclauses, ten representative standard control definitions were carefully selected to balance comprehensiveness with the feasibility of

TABLE 1. Example of system-specific requirements derived from a standard control definition.

Original standard control definition (ISO/IEC 27002)	Preprocessed, parameterized control template (annotated placeholders)	Relevant excerpt from the specification of the target system under development (AI4I4)	Instantiated requirements (examples generated by a human expert)
(8.2 g) regularly, and after any organizational change, reviewing users working with privileged access rights in order to verify if their duties, roles, responsibilities, and competence still qualify them for working with privileged access rights.	(#6) The {organization} shall regularly review {users} with privileged access rights to ensure that their {duties}, {roles}, {responsibilities}, and {competence} still justify such access.	These use cases are supported by datastores […]: Requisitions Data Store [... used by] Access Control Manager [...]. Supply Vehicles Registry [... used by] Access Control Manager and Dock Manager [...]. Warehouse Inventory [... used by] Dock Manager [...]. The Access Gate [...] enforces the authorization of all incoming and outgoing traffic [...]. It is managed by an Access Control Manager [...]. Factory Employees use a front-end application [...] to carry out tasks [...]. The interface application will implement different functions depending on the responsibilities and role of the current user.	(H.3.1) Logical access controls must be implemented to isolate Access Control data (such as schedules and vehicle registration data; stored in the Requisitions Data Store) from Dock Management data (such as unloading confirmations; stored in the Supply Vehicles Registry) and from Warehouse Management data (such as material stock and movements; stored in the Warehouse Inventory) (H.3.2) Physical access controls must be implemented at the premises where the physical systems that support the services are located. (H.3.3) Controls must be implemented that prevent access to any system or registry through any means different from the dedicated end-user front-end or the subsystem designed for such purpose.

the detailed manual judgment required in later stages of our process.

2) TARGET SYSTEM SPECIFICATION

The specification of the target system under development is provided as a plain-text document that presents an overview of the system, containing enough detail to make it feasible to generate specific cybersecurity requirements. In our case, we use: 1) the functional requirements of the system (specified according to a use-case model), 2) system constraints, and 3) a textual description of the domain model of the system context that describes the entities involved and their relationships.

For validation purposes, our work leverages a constructed example that provides the specification of a fictitious yet realistic example system, AI4I4 – Automated Identification for Industry 4.0, created as a research testbed for the evaluation of LLMs in software engineering tasks. This target system example showcases the integration of Internet of Things (IoT) technologies into smart manufacturing contexts, particularly to optimize logistics processes in a vehicle manufacturing plant. The system focuses on the use of Automatic Identification and Data Capture (AIDC) technologies to enhance material and product movement processes, from the reception of parts and raw materials to the delivery of finished vehicles to a loading dock adjacent to a logistics terminal next to the factory.

Although AI4I4 is a synthetic, purpose-built system example, it has been deliberately designed to emulate a realistic Industry 4.0 automotive manufacturing environment supported by AIDC (Automatic Identification and Data Capture) and IoT (Internet of Things) technologies. Its artifacts include a 5300-word system specification designed to enable focused generation of cybersecurity requirements, by providing a detailed description of the system's use cases, components, interfaces, and interactions, as well as the context in which the system operates.

The AI4I4 system is divided into three main subsystems that mirror the stages of the manufacturing processes it addresses: 1) inbound logistics and component reception, 2) vehicle assembly, and 3) outbound logistics and vehicle delivery. Each subsystem implements processes and requirements that are tailored to the tasks carried out at the corresponding stage. AI4I4 relies on middleware IoT platforms to connect sensors and actuators, manage emulators, and ensure the integration of external systems. The specification describes multiple actors and organizations, structured data stores and business artifacts, stepwise operational use cases, and cyber-physical interactions that together yield a dense network of information assets and trust boundaries. This provides functional and organizational complexity sufficient to exercise controls related to identification, access control, integrity, availability, traceability, and supply chain, and to test the ability of LLMs to apply them consistently across multiple subsystems and stakeholders. From the standpoint of our LLM-based evaluation, some features of AI4I4 make it a deliberately challenging but appropriate testbed. The tight coupling between digital records and physical operations makes it a demanding test of the models' ability to scope and distinguish such controls correctly, for example, between requirements on access to information assets (including computing equipment) and requirements on access to premises and manufacturing or logistics processes (which lie outside the scope of the system of interest). Likewise, the involvement of external actors through nonstandard interaction patterns that go beyond canonical textbook scenarios

(e.g., dealers querying the status of specific orders) challenges the models' capacity to tailor and instantiate standard controls across organizational boundaries.

3) SYSTEM-SPECIFIC CYBERSECURITY REQUIREMENTS

Given these inputs, the task outputs a list of specific, concrete cybersecurity requirements that the target system must meet, consequently expressed in the system's own context and vocabulary. Each generated requirement is presented in a structured format that includes:

- Name: a concise phrase that summarizes the requirement.
- Description: a fuller explanation of what the requirement entails.
- Control template, a reference (for traceability) to the originating control template from which the requirement is derived.
- Rationale: justification of the applicability (or lack thereof) of each control to the target system under development, and the mapping from specific entities of the target system domain back to the placeholders in the parameterized control template; ensuring that each requirement is grounded in the context of both the standard and the target system.

These requirements shall be generated in a coherent and homogeneous way to facilitate their implementation, and should be specific, measurable, achievable, relevant, and time-bound (also known as S.M.A.R.T. criteria).

C. DETAILED PROCESS APPROACH: DERIVATION OF SYSTEM-SPECIFIC REQUIREMENTS

Going deeper into how this task is approached in practice, we consider that the derivation of system-specific requirements from standard control definitions, whether performed by a human or by an automated solution, always follows a three-step process that includes (Figure 2): 1) the classification of (pre-annotated) control templates with respect to their applicability to the target system under development, 2) the identification of domain elements from the context of the system that map to placeholders in the text of the annotated, parameterized control template, and 3) the instantiation of system-specific requirements. As explained above, prior to these steps, each standard control definition has been preprocessed, by rewriting it as a parameterized template, annotated with marks of generic placeholders to be instantiated with system-specific values during requirement generation.

The LLM pipeline is invoked on a per-template basis: in each call, the model receives a single parameterized control template together with the relevant system artifacts as context. The output explicitly records, for every generated requirement, the full text of the control template being instantiated and a short rationale describing why the control is applicable and which system entities instantiate each placeholder in the template. This design preserves traceability from each LLM-generated requirement back to its originating

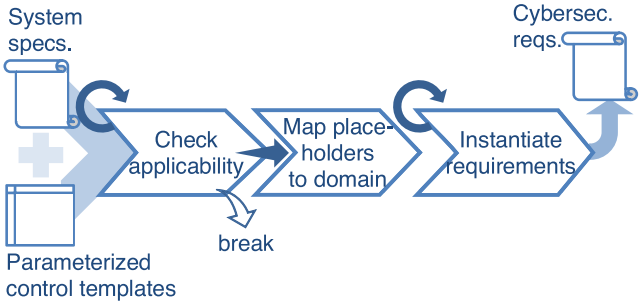


FIGURE 2. Process of deriving cybersecurity requirements from parameterized control templates.

control template. In turn, in our pipeline, each control template is tagged with its corresponding ISO/IEC 27002:2022 clause identifier in prompt input; Table 1 shows an example of this mapping (which is documented for all the requirements in the accompanying reproducible-research artifacts). Taken together, these elements enable every LLM-generated requirement to be traced back to its source clause in the standard.

1) APPLICABILITY CHECK

The initial step in the cybersecurity requirement generation process involves classifying the parameterized control templates (derived from standard control definitions), depending on their relevance to the target system. This step aims to determine which controls are applicable to the target system under development and should be instantiated into system-specific requirements. The classification process is guided by the textual specification of the system and the parameterized control templates themselves, enabling the filtering of irrelevant or inapplicable controls and focusing on those that are essential in the scope of the system.

This step becomes particularly important when dealing with standards that involve numerous controls, where not all may apply to the target system. By filtering out inapplicable controls regarding the characteristics of a given system, we prevent the generation process from being forced to produce irrelevant requirements.

2) PLACEHOLDERS–DOMAIN MAPPING

The second step in the cybersecurity requirements generation process involves identifying domain elements within the target system specification that are relevant to the (already filtered) parameterized control templates. Domain elements refer to specific entities, processes, technologies, or environments that are mentioned in the system specification and which are essential to instantiate the control templates in the context of the target system.

This step, whether performed by a human or by an automated solution, serves the purpose of identifying all the elements in the context of the target system under development that are necessary or related to the parameterized control template, ensuring that not only those elements that

are most evident are considered, but also others that may be less obvious but are equally relevant.

3) SYSTEM-SPECIFIC REQUIREMENTS INSTANTIATION

The final step in the process is the generation of the system-specific instantiated requirements: in a nutshell, this step can be summarized as creating a requirement for each tuple of domain elements that can play the role of the placeholder elements in the parameterized control template. This leverages the previously identified domain elements to instantiate the standard control definitions into system-specific requirements. The generated requirements are formatted in a coherent and homogeneous way, following a structured format (as shown in Section IV-B) that includes the name, details, rationale, and traceability to the control template instantiated. This includes a detailed justification of the relationship that exists between the control definition and the domain elements in the target system that yield this requirement.

D. SELECTION OF LLMS UNDER STUDY

Our work investigates how state-of-the-art large-language models (LLMs) can generate cybersecurity requirements when prompted in a non-interactive mode—that is, through a fully preprogrammed, single-turn interaction, or brief multi-turn exchanges with no human intervention between turns—and without resorting to external tools. Limiting ourselves to prompt-based use mirrors the realistic workflow of requirements engineers who employ models out-of-the-box, without any additional fine-tuning. Accordingly, we considered only frontier, decoder-only, instruction-following, non-agentic models accessible through widely available APIs; and excluded multimodal, code-specialized, or domain-specific variants, concentrating instead on general-purpose text generation. We also emphasize open-weights releases for their auditability and the possibility of confidential on-premises deployment.

Candidate models were drawn from the Hugging Face Open LLM Leaderboard [73] snapshot dated 15 September 2024. We used its composite score (average across several text-only benchmarks) but purposely omitted the MATH metric, which is tangential to requirements generation. The detailed query can be checked on the reproducible research repository. To narrow the list, we applied four sequential filters:

- Original provenance: only releases officially maintained by the original model authors, discarding third-party or community fine-tunings and tweaked re-uploads.
- Frontier status: when multiple versions exist, select the top-performing (usually largest and most recent) release to capture the state of the art.
- Geographic diversity: ensure representation from Europe, the United States, and China.
- Architectural diversity: at equal rank, prefer models that offer distinct design choices.

The final model set includes three open models (two using a dense transformer architecture and one using a sparse mixture-of-experts) and a leading proprietary alternative—providing sufficient breadth to analyze LLM behavior in prompt-based cybersecurity-requirements generation:

- Meta-LLaMa-3.1-405B-Instruct-v1.0: the largest publicly released dense model from Meta (not listed on the leaderboard but, according to Meta, it outperforms the 70 B variant that was included there); it offers frontier-scale capacity.
- Qwen2-72B-Instruct: adds linguistic and cultural coverage from a Chinese-developed model.
- Mixtral-8 $\times$ 22B-Instruct-v0.1: captures a sparse Mixture-of-Experts architecture and provides European provenance.
- GPT-4-Turbo-2024-04-09: a widely available closed model regarded as the state of the art at the time of our evaluation; it serves as a baseline.

Practical constraints on human-reviewer capacity—as the outputs of each model require manual judgment—restricted the evaluation to these four primary models rather than a larger set of weaker or closely related variants that would increase the evaluation cost without substantially enriching the range of observed behaviors. We also used two lighter variants exclusively for low-stakes formatting tasks (e.g., converting raw lists to JavaScript Object Notation - JSON): Meta-LLaMa-3.1-70B-Instruct and GPT-4o-mini-2024-07-18. Their smaller computational footprint may yield substantial cost savings in practice without undermining the validity of our core evaluation.

Independent evidence suggests that these families are also among the most widely used in enterprise practice. In a 2024 survey of 70 enterprise AI decision makers conducted by Andreessen Horowitz [85], 100% of respondents reported using OpenAI models, while 41% reported using LLaMa-based models and 17% using Mistral models, predominantly in testing environments (30% and 14% of respondents, respectively, reported testing these families). Complementing this, usage rankings from the OpenRouter platform [86] likewise show that the providers behind our selected models (OpenAI, Meta LLaMa, Mistral AI, and Qwen) together account for a meaningful share of tokens served—around 13.5% of cumulative traffic between July 2024 and June 2025 and close to one third of monthly traffic by our cutoff date—placing them among the more frequently used options on that platform. Table 11 in Appendix A further summarizes how these models rank across multiple public leaderboards and evaluation suites beyond the Hugging Face Open LLM Leaderboard.

E. PROMPTING APPROACHES

As LLMs are controlled through natural language instructions (prompts), the quality of the results depends quite a lot on the specific instructions provided. When LLMs are queried in an interactive fashion, subsequent user messages

may help achieve better responses; however, in scenarios where a single-turn query is reused verbatim—aside from run-time parameter substitution, as in our case—the result quality hinges entirely on prompt design. In this sense, both systematic research and trial-and-error practice have come up with a plethora of systematic techniques to achieve the desired results by appropriately crafting the prompts—an approach often termed "prompt engineering". Different sources have compiled catalogs of prompting patterns [60], [68], [87], [88], [89], [90], [91], [92] that steer the behavior of LLMs and help make the most of their capabilities.

Henceforth, in order to evaluate LLM capabilities, we must apply best-available prompting practices—under-specified prompts would not isolate model capability. Thus, we have leveraged quite a lot of these patterns to improve the quality of our results. The prompting approaches described herein set the experimental testbed that operationalizes our evaluation framework, enabling different models and configurations to be systematically compared on the same requirements-generation task under controlled conditions. We may group the prompting techniques employed depending on whether they are applied to an individual prompt or affect the overall organization of several requests to the LLM. The exact prompt contents we have employed can be found in the reproducible research repository.

1) INTRA-PROMPT TECHNIQUES

We depart from the overall prompt structure of Instruction – Context – Input data – Output format indicator. In our task, the instruction is the requirements instantiation task, the inputs are the parameterized standard requirements, and the final output format is a JSON list of annotated and instantiated requirements. In our case, we have switched the order of the last two elements, as it seemed that the LLM got "lost" when placing the longest input (the description of the system) after a shorter element (the standard parameterized requirement). Improving the prompt may affect any of those parts.

We have subjected our prompts to a manual adjustment process that includes the following techniques:

- Persona / Role Prompting: Start the conversation with a system message that fixes the model's persona, expertise, and tone.
- Applicability Filters: Insert an explicit check (e.g., "*If the requirement is not applicable, say so*") so the model can refuse or skip irrelevant items, reducing hallucinations.
- Few-shot Prompting: Supply one or more input-output exemplars so that the model infers the pattern before handling the real query. We have applied this in the model configuration L5 (see Table 2 below).
- Zero-shot Prompting: Give only the task instructions and trust the model to generalize without examples.
- Negative Prompts: State explicitly what must not appear (e.g., "*Don't mention organization names*"), guiding the model away from unwanted content.
- Meta Language Creation: Invent a mini domain-specific language or annotation scheme within the prompt so both the user and the model share shorthand semantics.
- Template Prompting: Create a reusable skeleton that contains placeholders (e.g., "*{system_spec}*") to be filled at runtime.
- Delimited-block Prompting: Use clear delimiters (triple quotes, XML tags, etc.) so that the model can unambiguously see where quoted text starts and ends.
- Instruction / Guideline Prompting: Enumerate explicit rules ("*Guidelines: ...*") that constrain format, style, or content.
- Golden Chain-of-Thought: Embed ideal reasoning steps that the model can imitate while only exposing the final answer.
- Explanations: Ask the model to justify or explain its answer, thus improving transparency and allowing verification.

2) REQUEST ROUTING TECHNIQUES

The following patterns, by contrast, operate across a chain of prompts rather than within one; routing and combining inputs and outputs from one request to LLMs to another in a chain of subsequent calls:

- Divide-and-Conquer: Break a large problem into subprompts, then merge the partial answers. In particular, we analyze each parameterized control template separately: instead of one big prompt, we loop over controls one by one—mirroring how a human assistant tackles tasks sequentially and keeps context short.
- Prompt Chaining: Split a complex job into ordered prompts whose outputs are fed into subsequent prompts, building an answer incrementally. Thus, each step in the process (see Section IV-C) is carried out by a separate prompt (except in configuration L4, see Table 2 below).
- Feature Augmentation: Inject entities or facts produced in earlier steps into later prompts to enrich context and accuracy.
- Rebroadcasting / Feed-Forward: Feed the model's previous output back into a fresh prompt to refine, summarize, or transform it further.
- Inter-model Interaction: Employ smaller or specialized models for subtasks such as formatting, and pass their output to the main LLM.
- Repeated Instructions / Resampling: Reissue the same request multiple times to harvest diverse candidates. We have applied this in one LLM configuration (L4, see Table 2), where we coalesce the results of parallel runs.
- Differential Prompting: Run controlled prompt variants to spot how small wording changes affect outputs; which is useful for A/B exploration or safety checks. Beyond iterative prompt refinement via trial and error, we have systematically analyzed the effects of removing several prompting techniques (see Section V-C below).

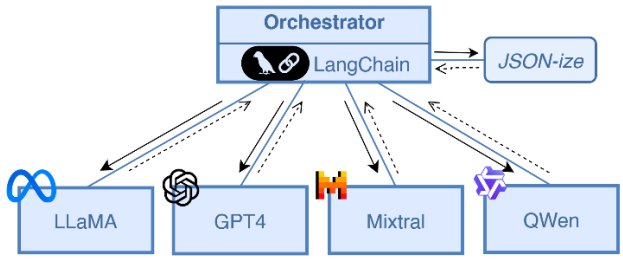


FIGURE 3. Control architecture of the testbed setup.

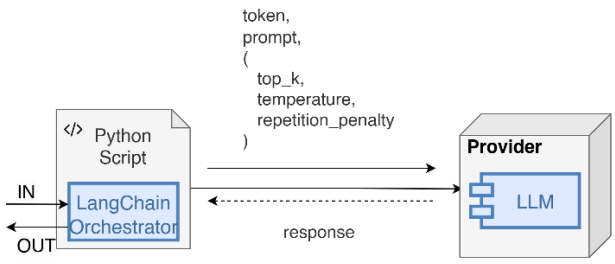


FIGURE 4. Testbed deployment and message passing.

TABLE 2. Model configurations.

Config. Id.	Model	Chain steps	Hyperparameters	
			temperature	top-*p*
L0	LLaMa 3.1 405B	3-step	Base	Base
L1	LLaMa 3.1 405B	3-step	Lower	Lower
L2	LLaMa 3.1 405B	3-step	Higher	Base
L3	LLaMa 3.1 405B	Branch-and-coalesce	Base	Base
L4	LLaMa 3.1 405B	All-in-one	Base	Base
L5	LLaMa 3.1 405B	Few-shot	Higher	Base
Q0	Qwen-2 72B	3-step	Base	Base
Q1	Qwen-2 72B	3-step	Lower	Lower
Q2	Qwen-2 72B	3-step	Higher	Base
M0	Mixtral 8x22B	3-step	Base	Base
M1	Mixtral 8x22B	3-step	Lower	Lower
M2	Mixtral 8x22B	3-step	Higher	Base
G0	GPT-4 Turbo	3-step	Base	Base
G1	GPT-4 Turbo	3-step	Lower	Lower
G2	GPT-4 Turbo	3-step	Higher	Base

In addition, GPT 4o mini and LLaMa 3.1 70B were used for the JSON-formatting step in some configurations of their respective greater models. Detailed hyperparameter values are available in Table 10 (Appendix A) and the replicable research package.

- Meta-prompt Generation: Have the LLM itself draft or refine prompts, iterating with user feedback until the prompt performs optimally.

F. TESTBED ARCHITECTURE

Once the models under evaluation have been chosen and the input prompts prepared for each process step and each model (and model configuration, if applicable), we have proceeded with the execution of the task by each LLM.

In order to run it, we have set up a *testbed* (Figure 3) where an *orchestrator* module iterates over each parameterized control template (see Section IV-B1), each step of the generation process (see Section IV-C), and each LLM and configuration, as defined for our evaluation (Table 2). Besides the requirements generation process itself, we also incorporate ancillary calls to lower-end LLMs to format the outputs as JSON structures, which facilitates subsequent analysis and integration into downstream tools or workflows.

The use of LangChain—an open-source orchestration framework to develop LLM-based applications—as a *façade* to the different models responds to three goals: automation, adaptability, and decoupling. This architectural approach enables the integration and coordination of multiple LLMs in our evaluation experiment, supports iterative and batched API calls, and allows experiments to be repeated seamlessly—facilitating trial-and-error exploration without burden and enabling fast iterations in prompt design and refinement.

From the *deployment perspective*, we should take into account that executing frontier LLMs is a computationally resource-intensive task that requires high-performance computing infrastructure. Therefore, our deployment leverages cloud-based LLM APIs to access the models, where each model is provisioned through a dedicated or a serverless endpoint in a remote cloud service provider, where the LLM is exposed through a controlled API (Figure 4). It should be noted that the LLM API endpoint here acts as a mere transport of the prompt (together with authentication and configuration data). The API parameters are themselves devoid of any functional semantics, while the task-specific instructions proper are encoded in natural language within the payload of the requests to such API—i.e., the prompt—.

Besides, key *hyperparameters*, such as temperature and top-*p* sampling, have been adjusted to balance creativity and coherence (see Section III-A above for definitions). In fact, the development of the testbed implied adapting the LLMs' hyperparameters to optimize the quality of the generated results: for example, a higher temperature has been used in the placeholder–domain mapping stage to encourage more creative associations to entities of the system, while lower temperatures have been employed in the requirement instantiation step, so as to maintain clarity and coherence. Different configurations were evaluated for each LLM with variations of hyperparameters (Table 2; detailed values for all the hyperparameters used in each step and model configuration can be found in Table 10, Appendix A). In all cases, we restrict ourselves to variations within standard operating regimes commonly used for these models (mid-range temperatures and typical top-*p* values) rather than extreme sampling settings.

The choice of LLM providers (Amazon Web Services Bedrock, Hugging Face, Mistral La Plateforme, and OpenAI) is based on the respective availability of each model among

their offerings, the ease of configuration, and usage pricing schemes. The use of LangChain also encapsulates access to instances of LLMs deployed in the diverse platform providers we use.

G. DATA ANALYSIS METHOD

Our goal is to determine the effectiveness of LLMs in the automated generation of cybersecurity requirements, focusing on two failure modes: omissions (missed requirements) and hallucinations (spurious or irrelevant requirements). We frame requirement generation as an *information retrieval task under an open-world assumption*: the set of potential cybersecurity requirements is unbounded, and our LLM may produce both relevant (to be scored as true positives) and spurious outputs (false positives) that lie outside any fixed reference corpus. We thus quantify LLM performance by measuring standard information retrieval metrics (precision and recall), adapted to this open-world setting, against manually expert-curated reference sets.

For this aim, we rely on two complementary *reference corpora*. First, the initial ground truth (manual baseline) comprises the cybersecurity requirements manually generated by human experts through the instantiation of parameterized control templates. Second, an expanded gold standard augments that baseline with every new, valid discovery ever produced by any LLM run. The latter is built using a pooled sampling strategy, akin to the pooling method employed in TREC (Text REtrieval Conference) evaluations [93], [94]: we aggregate the candidate requirements returned by all LLM configurations and runs, and then have experts judge their validity to add them to those in the manual baseline. This increases the likelihood that the expanded gold standard captures any requirement that a model could plausibly surface. Although this new set is still necessarily incomplete, we may consider it a good approximation reference set, especially as we focus on the comparison between the automated and the human approaches, rather than on the absolute values of each.

Because outputs from different sources may vary in granularity, we have manually coded a pair of *traceability tables* for each LLM configuration, which map from human- to LLM-generated requirements and vice versa, under which a variety of circumstances have been considered:

- True positives: a complete match between a manual and an automated requirement, accurately identified by the LLM.
- False negatives (omissions): requirements manually identified by the humans but entirely overlooked by the LLM.
- Partial coverage: situations where an automated requirement partially matches a manual requirement (e.g., specifying multi-factor authentication (MFA) only for employees instead of all users).
- Aggregate joint coverage: cases where multiple automated requirements collectively cover a single manual requirement.
- Multiple coverage: instances where a single automated requirement addresses multiple manual requirements.
- Discoveries: new, relevant requirements identified only by the automated method, which add value to the overall requirements generation process and are incorporated into the expanded gold standard.
- Hallucinations: incorrect or irrelevant requirements generated by the LLM, potentially introducing confusion or unnecessary complexity.
- Excess coverage: Automated requirements that go in part beyond their manual counterparts, covering additional aspects. The excess is coded as partial discovery or partial hallucination, as appropriate.
- Mixed cases: Complex scenarios that combine several categories mentioned above.

All categorizations in these tables were produced by the expert panel using a predefined coding rubric and consensus discussion. To reduce ambiguity and limit partial matches, coders were instructed to provide fine-grained, near-atomic requirements, so that each expresses a single, coherent security decision whenever possible.

From that mapping, we have defined performance evaluation criteria based on information retrieval relevance judgment metrics: precision and recall.

Precision (relevance) measures how many requirements generated by the LLM are genuinely applicable. It is computed as the ratio of reference requirements correctly identified by the LLM to the total number of LLM-generated requirements. To isolate precision from verbosity, true positives are *anchored to a reference set*—either the manual baseline ground truth or the expanded gold standard—. Formally, $\text{Precision} = N_{\text{covered}} / (N_{\text{covered}} + N_{\text{hallucinations}})$, where N_{covered} is the number of distinct requirements from the reference set that are matched by at least one LLM output, and $N_{\text{hallucinations}}$ is the count of LLM outputs judged entirely irrelevant or incorrect. That is, whether an LLM yields three fine-grained requirements or one coarse requirement for the same reference requirement, it contributes exactly one "hit" to both the numerator and the denominator. By counting each covered reference requirement only once as a true positive—regardless of how many LLM items point to it—and by counting each hallucination individually, this formulation eliminates the bias that might arise from different granularity in outputs among LLMs. Partial matches are counted twice, in both N_{covered} and $N_{\text{hallucinations}}$ (to reflect the fact that they denote one hit and one miss), and discoveries only enter N_{covered} when using the expanded gold standard (they do not exist in the manual baseline).

Recall (relative completeness) is also necessarily measured *with respect to* a reference set, because the full universe of valid requirements is unknown in practice. When using the manual baseline, recall tells us what proportion of human-generated requirements the LLM reproduces. When the expanded gold standard is used, our focus shifts to the recall gain delivered by the LLM over the manual process: the gain in the ratio between the LLM's and the manual process's

recalls against the expanded gold standard. This ratio tells us how much additional coverage of the known relevant content was captured by the LLM beyond human results (while still discounting hallucinations), effectively assuming that the union of all findings through all methods is our best approximation to the relevant items available.

To account for the ambiguity introduced by partial coverage cases, we carried out a *sensitivity analysis* that recalculates recall under two bounding assumptions. In a conservative (worst case) scenario, every partial match is treated as a miss and thus excluded from the true positive counts. In a liberal (best case) scenario, partial matches count as full hits, presuming that any degree of relevance suffices. For precision purposes, no sensitivity analysis is needed, given that partial matches are always computed as true positives, regardless of how far we deem that they match some human-generated requirement.

Whereas a conventional route to *statistical validation* would require several runs of the evaluation experiment, doing so would be prohibitively expensive in terms of human effort. Instead, we model each requirement decision as an independent Bernoulli trial: for recall, each requirement in the reference set is a trial whose outcome is a success if the LLM retrieves it; for precision, each requirement produced by the LLM is a trial whose outcome is success if experts validate it. Aggregating these trials yields binomial success counts, from which we estimate the underlying probabilities and their confidence intervals for a modest significance level α set to 0.1, given the exploratory nature of our evaluation experiment. Because these proportions can involve small sample sizes and extreme values, we compute Wilson's binomial proportion score intervals (1), which are particularly appropriate for binomial data (with better coverage and shorter lengths than the normal approximation intervals under such conditions),

$$\frac{1}{1+\frac{z^2}{n}} \cdot \left(\hat{p} + \frac{z^2}{2n} \mp z \cdot \sqrt{\frac{\hat{p}\left(1-\hat{p}\right)}{n} + \frac{z^2}{4n^2}} \right) \tag{1}$$

where $\hat{p}$ represents the estimation of our quantity of interest (namely, precision and recall), n is the total number of requirements (for precision, all those generated by a given LLM, whether valid or not; and for recall, all in the expanded gold standard), and z is the half-width interval of a standard normal corresponding to the desired confidence level $(1-\alpha)$.

This is supplemented with a *consistency analysis*, in which we examine run-to-run stability within and across models. Recognizing that LLM outputs may vary across runs with identical prompts (because of their inherent stochasticity), we issued the same prompt multiple times under fixed settings for each base configuration (L0, Q0, M0, G0) and for the iterations of the coalesced-output configuration L3, obtaining four independent generations per model configuration. For each model configuration M_C, we computed within-model agreement $\mathrm{Ag}(M_C)$ (2) using the mean of the pairwise Jaccard similarity indices (size of the intersection over size of the union) between the resulting requirement sets, calculated for each of the six unordered run pairs resulting from four runs, and then averaged, yielding a reproducibility score between 0 (no overlap) and 1 (identical outputs).

$$\mathrm{Ag}\left(M_C\right) = \frac{1}{\binom{n}{2}} \sum_{i=1}^{n} \sum\nolimits_{j=i+1}^{n} \frac{\left|\left\{\mathrm{req}_{M_c,\mathrm{run}_i}\right\} \cap \left\{\mathrm{req}_{M_c,\mathrm{run}_j}\right\}\right|}{\left|\left\{\mathrm{req}_{M_c,\mathrm{run}_i}\right\} \cup \left\{\mathrm{req}_{M_c,\mathrm{run}_j}\right\}\right|} \tag{2}$$

where $\left\{\mathrm{req}_{M_c,\mathrm{run}_i}\right\}$ denotes the set of requirements produced by a given model configuration M_C in run i. To distinguish variability in correct coverage from variability in hallucinations, we computed this agreement both including all generated requirements and after removing hallucinated items. In addition, we estimate between-model agreement by computing Jaccard similarities across runs belonging to the four base configurations and reporting their mean; this provides an aggregate measure of how much of the overall variation is due to stochasticity within a model versus systematic differences across models.

In order to synthesize precision and recall into a single metric, we use the Fβ *score*, a weighted harmonic mean of both aspects that allows us to emphasize either of them using a weight equal to β^2. When generating cybersecurity requirements, the cost of a missed control is far higher than the cost of reviewing an extra—possibly irrelevant or hallucinated—requirement: Under a human-in-the-loop workflow, a single omission may force engineers to painstakingly trawl every parameterized control template against every system element to spot what is missing, thus eroding the automation benefit. In contrast, a false positive is easier to spot and discard. Consequently, we set $\beta = 2$, placing four times as much weight on recall as on precision, and making the score more sensitive to improvements in coverage while still penalizing spurious additions. When the expanded gold standard is used, we complement the conventional F2 score by computing a relative F2 that replaces recall with the recall ratio (LLM recall divided by human recall). This nonstandard but analogous harmonic mean rewards models only when they both preserve high precision and achieve a tangible gain over human coverage, thus retaining the original spirit of F-scores while focusing straight on the incremental benefit of automation.

For completeness, we will also report other F-scores against the human baseline: the traditional F1 ($\beta = 1$), which gives equal weight to recall and precision, and F0.5 ($\beta = 0.5$), which moderately emphasizes precision. Nevertheless, as will be seen (Table 4 below), in our results, recall happens to be consistently lower than precision for all configurations; thus, placing more weight on recall yields lower composite scores. Hence, F2 systematically takes the lowest value and constitutes the most stringent of the three F-score metrics—while being more appropriate in our recall-critical scenario.

V. RESULTS

After having run the model configurations introduced above (Table 2), results have been analyzed against two com-

plementary reference sets, both generated from the same 10 parameterized, abstract control templates: 1) a ground truth (manual baseline) of 30 requirements produced by human experts, and 2) an expanded gold standard (GS) of 76 requirements obtained by pooling those generated by any human or LLM, and vetting them through expert review.

We first report (Section V-A) performance metrics—precision, recall, recall gain and F-scores—of the base configurations of each model, to show how each succeeds in reproducing the baseline and how much fresh coverage it contributes to the expanded gold standard. Subsequent subsections analyze the influence of the model, prompt design, and sampling hyperparameters, and quantify run-to-run variability.

A. MODEL ACCURACY AND BENCHMARK COMPARISON

When comparing the various LLM configurations with the human-generated reference set (H), L0 (LLaMa, base configuration) emerged as the most performant model across several key metrics. It registered an F2 score between 0.47 and 0.53 (worst/best cases), outperforming all other configurations under identical evaluation conditions (refer to Table 3, Table 4, Figure 5, and Figure 6). In contrast, G0 (GPT-4, base configuration) recorded F2 scores between 0.26 and 0.32, with a lower recall and slightly higher hallucination count: L0 managed to recover 22 of the 30 cybersecurity requirements defined in the human baseline, introducing only 2 hallucinated items, while G0 recovered 16 of 30, with 5 hallucinations.

Against the expanded gold standard (GS), L0 maintains its lead. It covers (whether fully or partially) 35 of the 76 gold standard requirements, including 14 discoveries of additional unique items not present in the manual baseline. Its best-case relative F2 (designed to analyze the performance against the expanded gold standard, normalized relative to that of human experts) reaches 1.14, showing a mix of coverage that exceeds human experts—albeit not necessarily in the same direction, as revealed by discoveries—and a focus on truly relevant requirements, avoiding out-of-scope items or hallucinations. G0, in comparison, covers 21 items, discovering 5 new ones, and shows a relative F2 between 0.60 and 0.72.

Other models showed a range of performances. L1 and L2, (LLaMa configurations with alternative temperature values), achieve best-case F2 scores of 0.36 and 0.38, respectively. Among the Qwen and Mixtral contenders, each achieved its best result in the base configuration (Q0 and M0, respectively).

In practice, partial-coverage cases are relatively rare in our traceability tables: 14% of the judgments issued in the reference-anchored mappings (where each requirement in the reference set is checked against LLM outputs), and 2% in the reciprocal, candidate-anchored mappings. Consequently, the gap between worst-case and best-case values is modest and does not change which configurations consistently appear as the top contenders.

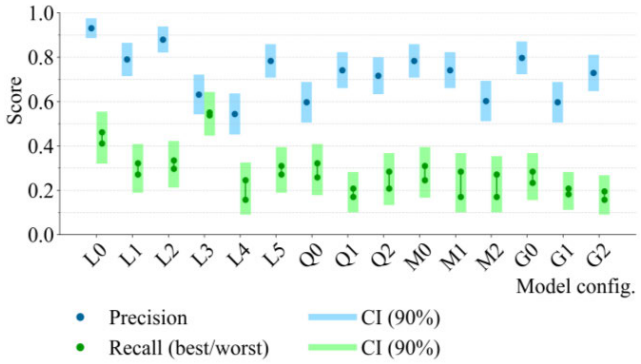


FIGURE 5. Precision and recall (best/worst case and 90% Wilson CIs) for model configurations against the expanded gold standard (n=76). Error bars show 90% CIs.

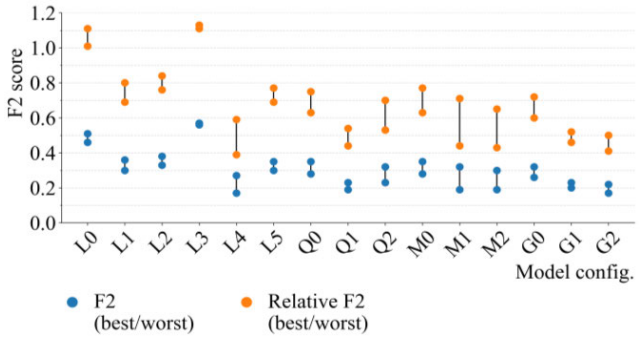


FIGURE 6. F2 against the expanded gold standard: absolute and relative (LLM recall / human recall in F2).

B. EFFECT OF HYPERPARAMETER TUNING

Across all model configurations, those identified as L0, Q0, M0, and G0 respectively yielded the highest performance under the tested settings (see Table 2 in Section IV above, for individual hyperparameter definitions and Table 4 for associated metrics; Figure 5 presents 90% confidence intervals for precision and recall, along with best–worst case sensitivity analysis). These base configurations utilized moderate values for temperature and top-p sampling, balancing diversity and determinism.

Within their respective models, the base configurations achieved the top recall values: L0 with 73%, Q0 with 67%, M1 with 60%, and G0 with 53%. Alternative configurations with increased temperature or top-p values, such as L2, Q2, and M2, tended to exhibit a higher number of hallucinations (e.g., Q2: 8, M2: 13) and a drop in precision (e.g., Q2: 67%, M2: 60%).

The impact of these hyperparameter changes was also reflected in the F2 scores, dropping to, e.g., 0.38 for L2 and 0.22 for G2, indicating degraded reliability in task performance. Additionally, the width of precision and recall confidence intervals increased with less constrained configurations, further supporting the stability achieved with moderate settings.

TABLE 3. Model Performance Summary (Manual Baseline).

Model config.	Human	L0	L1	L2	L3	L4	L5	Q0	Q1	Q2	M0	M1	M2	G0	G1	G2
Precision	100%	**92%**	75%	**85%**	50%	50%	73%	56%	72%	67%	70%	72%	58%	76%	57%	72%
Recall (worst)	100%	**60%**	47%	47%	**77%**	27%	43%	50%	33%	33%	30%	30%	33%	40%	37%	33%
Recall (best)	100%	**73%**	60%	57%	**80%**	50%	53%	67%	43%	53%	47%	60%	60%	53%	43%	43%

TABLE 4. Model performance summary (expanded gold standard).

Model config.	Human	L0	L1	L2	L3	L4	L5	Q0	Q1	Q2	M0	M1	M2	G0	G1	G2
Precision	100%	**95%**	80%	**89%**	64%	55%	79%	60%	75%	72%	79%	75%	61%	81%	60%	74%
Recall (worst)	39%	**42%**	26%	29%	**54%**	14%	26%	25%	16%	20%	24%	16%	16%	22%	17%	14%
Recall (best)	39%	**47%**	32%	33%	**55%**	24%	30%	32%	20%	28%	30%	28%	26%	28%	20%	18%
Recall gain (worst)	—	**+7%**	−33%	−27%	**+37%**	−63%	−33%	−37%	−60%	−50%	−40%	−60%	−60%	−43%	−57%	−63%
Recall gain (best)	—	**+20%**	−20%	−17%	**+40%**	−40%	−23%	−20%	−50%	−30%	−23%	−30%	−33%	−30%	−50%	−53%
F1 (worst)	0.57	**0.58**	0.40	0.44	**0.58**	0.23	0.40	0.35	0.26	0.31	0.36	0.26	0.25	0.35	0.27	0.24
F1 (best)	0.57	**0.63**	0.45	0.48	**0.59**	0.33	0.44	0.41	0.31	0.40	0.44	0.40	0.37	0.41	0.30	0.29
F2 (worst)	0.45	**0.47**	0.30	0.33	**0.56**	0.17	0.30	0.28	0.19	0.23	0.28	0.19	0.19	0.26	0.20	0.17
F2 (best)	0.45	**0.53**	0.36	0.38	**0.57**	0.27	0.35	0.35	0.23	0.32	0.35	0.32	0.30	0.32	0.23	0.22
Relative F2 (worst)	—	**1.04**	0.69	0.76	**1.11**	0.39	0.69	0.63	0.44	0.53	0.63	0.44	0.43	0.60	0.46	0.41
Relative F2 (best)	—	**1.14**	0.80	0.84	**1.13**	0.59	0.77	0.75	0.54	0.70	0.77	0.71	0.65	0.72	0.52	0.50
F0.5 (worst)	0.77	**0.76**	0.57	0.63	**0.61**	0.35	0.57	0.47	0.43	0.47	0.54	0.43	0.39	0.53	0.40	0.41
F0.5 (best)	0.77	**0.79**	0.61	0.66	**0.62**	0.43	0.60	0.51	0.48	0.55	0.60	0.56	0.48	0.58	0.43	0.46

TABLE 5. Comparison among prompt variations.

Model config. and variation	L0	L4	L0a	L0b
Precision	92%	50%	45%	44%
Recall (worst)	60%	27%	27%	17%
Recall (best)	73%	50%	47%	47%

C. PROMPT TUNING

The effect of prompt design was also investigated across different configurations. The base prompt pipeline (used in L0, Q0, M0, and G0) is structured around three passes that follow an explicit chain-of-thought; each prompt includes persona-based roles (e.g., "*you are a cybersecurity requirements engineer*"), repeated instructions for reinforcement, and requests for a rationale that supports the chosen domain mapping. These designs were consistently associated with higher recall and fewer hallucinations than alternative versions.

As an example, L0 under this structure resulted in 2 hallucinations and 73% recall. When simplifying the prompt through removal of one or more features, performance declined. Similar degradation trends were observed in Qwen and Mixtral series.

Three prompt variants were also evaluated in detail, namely:

- L4: a single-step, non-chain-of-thought prompt that bypasses intermediate reasoning steps, i.e., the LLM is asked to directly instantiate the parameterized templates, rather than having an explicit chain of steps whose responses feed the subsequent prompts.
- L0a: a variant without repeated instructions, thus limiting reinforcement effects
- L0b: a minimalistic prompt that omits persona and rationale components.

As summarized in Table 5, all three variants exhibited statistically significantly worse performance (against the human baseline ground truth) than the corresponding best prompt pipeline for the same LLM, and, in most cases, even lagged behind most configurations of the other models, both in terms of precision (hallucination) and recall (omission).

D. RESULT CONSISTENCY

To assess the stability of model outputs across repeated executions, we use the agreement measure $\mathrm{Ag}(M_C)$, defined in Section IV-G, based on the mean of pairwise Jaccard similarities across four independent runs per model configuration. Table 6 and Table 7 report these within-configuration agreement scores for the four base configurations (L0, Q0, M0, G0) and for the coalesced-output configuration L3 (which unifies outputs from four LLaMa generations into a single list), together with the corresponding between-configuration similarities.

TABLE 6. Jaccard index similarities (hallucinations included).

Model config.	L0	G0	M0	Q0	L3
L0	0.600	0.280	0.261	0.242	
G0		0.558	0.182	0.182	
M0			0.402	0.165	
Q0				0.529	
L3					0.564

TABLE 7. Jaccard index similarities (hallucinations excluded).

Model config.	L0	G0	M0	Q0	L3
L0	0.711	0.385	0.392	0.389	
G0		0.701	0.359	0.462	
M0			0.647	0.333	
Q0				0.788	
L3					0.730

Within each configuration, mean Jaccard similarity across runs (including hallucinations) lies in the range [0.402–0.600], whereas it increases to [0.647–0.788] once hallucinated requirements are removed. This indicates that the core set of valid requirements identified by a given configuration is relatively stable across runs, whereas hallucinations contribute disproportionately to within-model variability. By contrast, between-model Jaccard similarities remain substantially lower (around 0.219, or 0.387 without hallucinations), confirming that systematic differences between model families dominate the overall variation in generated requirements.

The coalesced-output configuration L3 achieved the highest recall at 80% and the top best-case F2 score of 0.57 (a +40% recall increase over human, yielding a relative F2 of 1.13). However, this configuration also recorded the highest hallucination count of 24, penalizing precision and illustrating the trade-off inherent in aggregating diverse outputs.

In terms of sensitivity to the number of partial matches, output variability among the best configurations was measurable but limited: L0's F2 scores ranged from 0.46 to 0.51, and G0's ranged from 0.26 to 0.32. This reflects stable performance under tuned settings, with additional reliability potentially gained from aggregation strategies such as those used in L3.

E. BREAKDOWN OF LLM MISTAKES

To better understand the nature of incorrect LLM outputs, we have further analyzed the nature of the hallucinations and omissions found. We have manually classified the hallucinated requirements identified in our experiments into five categories (Table 8):

- Template drift (38 instances) denotes hallucinations that are not valid instantiations of the specific parameterized control template being instantiated (even when they would be legitimate instantiations of other, unrelated ISO/IEC 27002 control definitions; for example, one requirement proposed generic network-encryption measures when instantiating a key-management control).
- Contextual misfit hallucinations (12) are requirements that might be reasonable in related systems but do not apply to the system as specified in the artifacts provided as input (e.g., protecting data flows that do not exist in the modeled architecture).
- Scope misalignment (26) captures requirements that fall outside the scope of the system as specified; a common subcase in our AI4I4 system involves requirements that concern the security of the broader physical facility (e.g., access control to production lines or the finished-car yard) rather than access control to the AI4I4 system itself.
- Responsibility leakage (3) refers to requirements that attempt to impose obligations on external organizations interacting with the system (e.g., in our case, freight forwarders), which lie outside the system's responsibility.
- Architectural infeasibility (47) covers requirements that are sensible in principle but cannot be enforced by the given architecture (e.g., isolating network traffic at a layer that the system does not control).

TABLE 8. Taxonomy of hallucinations.

Category of hallucinations \ Model	LLaMa	Qwen	Mixtral	GPT	Total
Template drift	9	18	1	10	38
Contextual misfit	9	3	0	1	13
Scope misalignment	20	5	1	0	26
Responsibility leakage	3	0	0	0	3
Architectural infeasibility	14	2	24	7	47
All hallucinations	55	28	26	18	127

The majority of hallucinations thus fall into overextending the scope of the system or the control, rather than introducing arbitrary content.

For completeness, we also examined the subset of human-coded requirements from the manually derived baseline that were most frequently omitted across models, configurations, and runs. By construction, this is a small subset: omissions are bounded by the size of the human baseline, and we focus on requirements that are rarely produced by any model or configuration. These omissions tend to cluster around less obvious aspects of our AI4I4 domain, such as controls involving the *Dealer* (an external business partner granted restricted access to certain system functions) and safeguards around automated identification carriers (e.g., Radio-frequency identification [RFID] tags and similar AIDC media used to encode vehicle or parts identifiers), whereas more obvious access-control requirements for the core system and its primary users are typically covered. Given the limited size of this subset, we did

not further sub-categorize omissions beyond this qualitative pattern.

VI. ANALYSIS AND DISCUSSION

In light of the research questions, we now bring together the quantitative results from our evaluation experiment, analyze the qualitative patterns that emerge across models and configurations, and discuss their implications for the role of LLMs in cybersecurity requirements engineering and for the validity and generalizability of our findings.

A. ANSWERS TO RESEARCH QUESTIONS

The analysis of our experimental results from the perspective of the research questions sheds light, from different perspectives, on how well LLMs perform in a nuanced and high-stakes task such as cybersecurity requirements generation.

Regarding RQ1, our findings indicate that LLMs can translate abstract control definitions extracted from system-independent cybersecurity standards (expressed as parameterized control templates) into concrete, actionable, and system-specific requirements when provided with the system's functional specification as context. Results show that the best LLM configurations achieve overall performance roughly comparable to the manual baseline (Table 3, Table 4, Figure 5, and Figure 6). For example, the LLaMa model's L0 configuration matches the human baseline on key performance metrics, with a relative F2 of 1.14. Taken together, these results suggest that LLMs can offer practical support for generating cybersecurity requirements. As there are very few borderline (partial) matches, resolving them with more sophisticated semantic-similarity thresholds would have only a limited effect on the aggregate metrics.

A deeper inspection reveals that humans and LLMs exhibit different strengths: human experts cover certain areas more reliably, while LLMs surface additional requirements that might otherwise be overlooked. Returning to the L0 configuration, while it misses 8 of the 30 requirements present in the manual baseline ground truth (73% recall), it also discovers 13 new requirements (not present in the manual baseline) grounded in the context of the target system, while producing only 2 hallucinations. We therefore regard humans and LLMs as complementary agents in a hybrid workflow, in which manually derived requirements are combined with LLM-generated candidates that engineers then review, prune, and validate.

RQ2.1 examines how frontier open-weights models compare with proprietary (closed) models. Somewhat surprisingly, the performance achieved by frontier open-weights models is, in our case, at least comparable to, and on some metrics better than, the proprietary baseline of the same model generation. In particular, LLaMa 3.1 405 B attains higher precision and recall than GPT 4 Turbo, while maintaining acceptable precision. LLaMa's outputs are more exploratory and creative in framing candidate requirements, which makes the model more likely to instantiate abstract control templates into concrete, system-specific requirements—while still keeping hallucinations under control. By contrast, the proprietary model we analyzed (GPT-4 Turbo) behaved more conservatively and often avoided aggressive instantiation of abstract terms—potentially to avoid over-committing—at the expense of recall and discovery of new requirements.

To situate these findings, we can contrast our domain-specific results with public leaderboards that aggregate broad capabilities: LMArena [95], the Hugging Face Open-LLM Leaderboard [96], and Stanford Center for Research on Foundation Models (CRFM) HELM Capabilities [77], [97]. Across these sources, LLaMa-3.1-405B consistently ranks above Qwen2-72B, and both above Mixtral-8 $\times$ 22B; with GPT-4-Turbo immediately following LLaMa-3.1-405B in the only leaderboard where all four models are compared (details are shown in Table 10, in Appendix A). This aligns with our observation that LLaMa-3.1-405B leads within our task at hand. Nonetheless, we caution against overinterpreting similarities or differences between our results and public ranks, given that: 1) not all four models appear in every leaderboard, limiting one-to-one comparisons; 2) task mixes and metrics differ among leaderboards and, within a given leaderboard model, ranks vary across tasks, hence a model that ranks lower overall may still excel at cybersecurity requirements generation (our recall-weighted F2 setting grounded in system context and control templates); and 3) in our results, several confidence intervals overlap, and within-model variability across configurations often exceeds between-model differences.

RQ2.2 addresses prompt design, and the results were unequivocal—prompting is not just a detail, but a cornerstone of LLM efficacy. Prompt structure has a material impact on outcomes: structuring prompts into a multi-step chain-of-thought reasoning pipeline, including clearly defined roles, and explicitly requiring rationales; all contributed significantly to output quality. Conversely, prompts lacking these elements resulted in significant losses in both precision and recall, leading to more hallucinations due to off-target requirements.

RQ2.3 asks whether LLM performance could be influenced by variations in hyperparameter settings. Empirical evidence supports this and aligns with the theoretically expected behavior of decoding hyperparameters. Within the standard operating range that we explore, changes in temperature and top-p systematically shift the balance between omissions and hallucinations. Mid-range values for these generation hyperparameters generally yield the best compromise, whereas more conservative configurations (lower temperature, tighter sampling) encourage cautious behavior that tends to reduce hallucinations at the cost of omitting some less obvious but valid requirements. Conversely, more exploratory configurations (higher temperature, broader sampling) increase diversity in the outputs and can improve recall, but they also introduce additional out-of-scope or hallucinated requirements and can cause semantic drift leading to

vague or overly imaginative statements that diverge from the cybersecurity-specific framing. Precisely locating an application-specific ''sweet spot'' for each model would require a denser hyperparameter sweep and more runs per configuration, which we leave for future work.

RQ2.4 focuses on the variability of model outputs across repeated executions using the same input. While stochasticity is intrinsic to LLMs, our multi-model analysis reveals a nuanced picture. Within a fixed configuration, repeated runs show a meaningful level of reproducibility: once hallucinations are discounted, the overlap of valid requirements across runs is consistently high, indicating that each configuration tends to yield a relatively stable ''core'' coverage pattern. Most within-model variation stems from hallucinations, which tend to appear and disappear across runs without affecting that core. By contrast, between-model differences are substantially larger than within-model differences, underscoring that architectural and training choices have a stronger impact on the generated requirement sets than sampling noise alone.

Taken together, these answers sketch out a detailed and practical roadmap for deploying LLMs in cybersecurity contexts. While performance is already promising, full effectiveness hinges on thoughtful prompt engineering, disciplined parameter tuning, and potentially, ensemble strategies—although this last approach entails a review overhead that needs to be absorbed.

B. ADDITIONAL RELEVANT FINDINGS

Beyond the numeric metrics provided in Section V, delving deeper into the behavioral patterns exhibited by the models reveals further nuances. For example, different configurations of any given model tend to behave similarly in terms of both strengths and limitations. By way of example, Qwen outputs not only yielded similar hallucinations every time, but also consistently overlooked the same subset of human-written requirements. This suggests that some blind spots are architectural rather than incidental.

A second prominent insight emerged from the way some models reorganized the set of requirements they produce. Instead of anchoring requirements to explicit agents (e.g., ''users'' or ''admins''), they recentered them around actions, objects, or roles. This orthogonal reframing provides just as valid a way of partitioning and organizing requirements, which points to the models' capacity for abstraction, while at the same time hinders comparability of results between one source of requirements and another (whether LLM or human).

Imagination in the phrasing of the requirements varied significantly across models. Although LLaMa and Mixtral were sometimes more creative, Qwen and GPT-4 often defaulted to conservative and familiar formulations that almost mimicked the original wording of the parameterized control template—even when guided by a high temperature hyperparameter. This restrained creativity might be viewed as a safeguard against hallucination, but it also limits the model's capacity to produce novel, yet valid insights.

Some requirements posed particular challenges. Those that encapsulated nuanced, subjective judgments, or relied on the existence of organization-specific policies were consistently under-represented in LLM outputs. This suggests that LLMs are still most effective when dealing with explicit and concrete input data.

Moreover, models frequently introduced quantification—even when discouraged. Requirements containing measurable parameters (e.g., ''*review every 30 days*'') were common, often accompanied by arbitrary periodicities or performance indicators. While such suggestions may occasionally align with best practice, they also reflect arbitrary assumptions that respond to neither the description of the target system under development nor the parameterized control templates provided as inputs.

Finally, hallucinations were found to vary from run to run—models did not consistently reproduce the same false positives. Taken over repeated runs, this suggests a lack of systematic self-correction capabilities. Some hallucinated requirements, while plausible in syntax, turned out to be meaningless combinations of buzzwords when examined more closely. Especially upon poorly framed prompts, models frequently misunderstood their task boundary, proposing requirements that were either too generic or completely disconnected from cybersecurity concerns. This misalignment underscores the role of prompt clarity in directing LLM attention.

Nonetheless, the variability in hallucinations between runs can be an asset when outputs from different runs or models are aggregated. The raw coalesced L3 configuration shows that combining multiple runs can substantially enhance recall and stabilize the core set of valid requirements, albeit at the cost of additional hallucinations. This suggests that more selective, ensemble approaches could use run-to-run agreement to prioritize requirements that appear consistently across runs and treat run-specific items as lower-confidence candidates, thereby increasing coverage while keeping hallucinations under closer control.

C. OPERATIONAL ROLE OF LLMS

Our results indicate that LLMs can generate cybersecurity requirements that human experts do not propose, but they are not yet ready to be left unsupervised in security-critical engineering tasks. Consequently, LLM-generated requirements would be best used within a human-in-the-loop workflow as an ancillary source of candidate requirements: human engineers retain responsibility for ensuring that no critical requirement is omitted and for pruning hallucinated or out-of-scope suggestions.

In a production deployment of LLMs as cybersecurity requirements engineering assistants, we envision the following hybrid workflow. First, the requirements engineer manually derives an initial set of requirements from the system specification and applicable standards. In parallel,

the LLM is prompted with the same artifacts using our engineered configuration, producing additional candidate requirements. The LLM output is structured to include, for each candidate requirement, the originating control template and a short rationale, which both helps the engineer decide whether the requirement is pertinent or should be rejected as a hallucination, and supports explicit traceability to the source control. In a subsequent review step, the engineer inspects each LLM-suggested requirement, discards hallucinations or out-of-scope items, and retains or refines those that add value. Engineers would be trained to pay particular attention to the most frequent hallucination categories observed in our taxonomy (Section V-E), while also being cautious not to discard demanding but valid requirements solely due to implementation difficulty.

From a practical standpoint, omissions and hallucinations have different cost profiles. An omission corresponds to a valid requirement that is not proposed at all; even a small probability of such an omission forces engineers to revisit the full space of control templates and relevant system entities, turning omission detection into a costly search problem. The omissions encountered suggest that some of the less obvious, higher-difficulty requirements remain hard to elicit for current LLMs and would likely also challenge non-expert analysts, which further reinforces our positioning of LLMs as assistants that complement expert judgment rather than replace it.

A hallucination, by contrast, presents itself as a candidate requirement; once the candidate is on the table, deciding whether it is in scope and enforceable is a much cheaper decision problem. The hallucination taxonomy introduced above (Section V-E) suggests that most hallucinated requirements either lie clearly outside the system boundary (contextual misfit, scope misalignment, responsibility leakage) or are architecturally infeasible, and can therefore be discarded quickly by a requirements engineer knowledgeable about the target system under development. Template-drift cases are subtler: they often look like reasonable security requirements but do not instantiate the intended control template.

As LLM-generated candidates are to be reviewed under a human-in-the-loop workflow rather than used as automatic specifications, the residual cybersecurity risk is therefore less about accepting hallucinations—which mainly risks feature creep or misplaced effort—and more about over-discarding valid but demanding requirements as if they were hallucinations (e.g., labeling them as unrealistic). Therefore, we recommend that engineers do not reject LLM-generated requirements solely because of their implementation difficulty but instead prioritize all requirements as first-class citizens based on a balance between expected security benefits (risk reduction) and implementation effort.

This workflow abstracts, from the final stages of our controlled study, a copilot pattern: LLMs propose and prioritize system-level cybersecurity requirements derived from standards and the system description, increasing coverage and surfacing non-obvious candidates, while human experts arbitrate which requirements are adopted into the specification and remain responsible for realizing them through concrete technical and organizational measures.

With respect to established processes for security requirements engineering, this envisioned workflow is not a replacement for but a complement to them, sketching a conceptual integration pattern for how LLMs could act as copilots for security requirements engineering within existing processes. For instance, in frameworks such as SEI's SQUARE [98], an LLM assistant could support the elicitation of candidate security requirements from control catalogs and system descriptions (SQUARE Step 6) and the requirements inspection (Step 9), by proposing standards-linked requirements and helping reviewers spot omissions or ambiguities, while final acceptance remains with the human team. Similarly, in the Open Web Application Security Project (OWASP) Software Assurance Maturity Model (SAMM [99]) and related practices (e.g., Comprehensive, Lightweight Application Security Process - CLASP [100]) that call for deriving security requirements from functional requirements, organizational policies, and domain knowledge, our approach can serve as a copilot that translates baseline controls (e.g., from ISO/IEC 27002) into project-specific requirements, subject to the same quality criteria and approval workflows as manually authored requirements. In principle, this pattern could also support more continuous compliance activities, for example by using the same assistant to re-check evolving architectures and new projects against the applicable control set. This is also aligned with ISO/IEC 27002's own emphasis (clauses 5.8, 8.25 and 8.26) on capturing security requirements early in the development lifecycle; our method simply offers one possible mechanism to operationalize this expectation.

Besides, following the security-by-design paradigm, this workflow may help security requirements be treated as testable artifacts (linked to acceptance tests, static analysis checks, or compliance rules), enabling more continuous forms of assurance in which an assistant periodically re-instantiates controls as architectures evolve and flags divergences between the current system and the intended set of security requirements. Nonetheless, integration into concrete requirements-engineering environments (e.g., requirements management tools, issue trackers, or continuous integration/continuous deployment pipelines) and their evaluation with industrial stakeholders are thus left for future work.

While a full economic cost–benefit analysis lies outside the scope of our evaluation experiment, our experience may provide an indicative sense of orders of magnitude: total LLM usage costs for all runs remained in the low tens of euros, whereas constructing the manual baseline required several tens of person-hours of expert work (also aligned with CLASP effort estimations to 'document business-level and functional requirements for security' [100]). Under a realistic human-in-the-loop workflow, the dominant cost therefore remains human time: the marginal cost of using LLMs arises primarily from reviewing additional candidate requirements (including hallucinations), while the potential benefit lies

in the extra valid requirements they surface beyond what humans alone identify within the same timeframe. A rigorous quantification of this trade-off—e.g., cost per additional correctly instantiated control under different deployment scenarios and price points—is left for future work. This sweet spot is also likely to shift as LLM capabilities, effective cost per task point, and underlying training costs evolve.

D. SCOPE LIMITATIONS AND VALIDITY CONSTRAINTS

As an exploratory study of novel AI technologies, it is essential that we acknowledge any factors that may temper the interpretation and generalizability of our findings.

In this work, we report the results of an *empirical single-system benchmark on a constructed system specification*—a deliberate first step rather than a definitive yardstick. *External validity* is therefore our main limitation: statements about effectiveness must be read within the scope of this specific system, standard, and experimental setup, rather than as global claims. The English-language specification of AI4I4—a purpose-built yet realistic smart manufacturing scenario—provides an ad hoc, fit-for-purpose dataset for grounding our findings in this domain. Because this focused testbed is tailored for the study, it reduces the risk of contamination by data that may have been used during LLM pretraining, but it necessarily limits generalizability to other systems, domains, languages, and input modalities (e.g., diagrams processable by multimodal LLMs). Additionally, our cybersecurity requirements are derived from ten control definitions from ISO/IEC 27002.

This design is intentional: the evaluation method requires human experts to construct a manual baseline, adjudicate the correctness and relevance of each LLM-generated requirement and rationale, and code error categories at a fine-grained level. The associated effort imposes practical limits on the number of systems, standards, and controls that can be analyzed without compromising evaluation quality. Broader empirical studies—spanning diverse architectures, control families, and natural languages—are required to confirm whether observed performance patterns hold more universally.

Regarding population validity, the three researchers, acting as the human cybersecurity analysts, curated a high-quality manual baseline ground truth, approximating an upper bound on human performance. This ceiling benchmark approach intentionally trades ecological realism for a rigorous comparison point; in typical industry settings staffed by non-specialists, LLMs might yield disproportionately greater benefits. On a related topic, although our prompt engineering workflow was resource-intensive, this does not prevent plug-and-play replication, as our prompts are project-independent (they are fine-tuned to the task at hand, but not to any specific target system under development) and are made available in the reproducible research package. Thus, their reuse by practitioners does not require any added effort, effectively setting a baseline for the future, and hence ensuring that they can achieve comparable results in the automated approach.

Finally, the rapid pace of model development means that the September 2024 snapshots we employed may have already been superseded as frontier models advance. As such, model rankings should be interpreted as temporal snapshots rather than enduring hierarchies.

On *internal validity*, we must first and foremost acknowledge the inherent stochasticity in LLM outputs: repeated runs with the same prompt and configuration can yield different results. Although we have explored this effect through the aggregation of the output of several runs (in the L3 configuration), reproducibility remains a challenge unless multiple outputs are consistently reconciled.

Similarly, the manual construction of the baseline ground truth by a small expert panel—who also adjudicated LLM outputs against this baseline—inevitably introduces observer and confirmation bias and limits reproducibility: even under consensus procedures, repeated executions might yield subtly different decompositions of the requirements space. This baseline is intentionally designed as a ceiling on attainable human performance rather than a sample of typical practice, and disagreements between analysts are resolved with an emphasis on maximizing coverage (rather than inter-rater uniformity for codebook calibration): findings by a single coder enrich the diversity of requirements in the human baseline, and they do not hamper validity as long as they are validated by the rest ex post. This ceiling benchmark and coverage-oriented consensus are consistent with our emphasis on recall as the primary effectiveness metric, and are consistent with standard practice in Information Retrieval; however, they also make the evaluation more stringent and divert it from typical industrial competence. We also recognize that human analysts may miss certain cybersecurity requirements. To reduce the risk of omissions, we complement the manual baseline with an expanded gold standard that includes any LLM-discovered requirement subsequently validated by the experts—adhering to pooling best practice from Information Retrieval. This strategy increases coverage but cannot fully eliminate residual blind spots that might only be addressed by a broader expert panel—our relatively focused case, however, reduces the risk of pooling bias from large volumes of irrelevant results. Another potential issue is related to the specific choice of the prompts used: The complexity of prompting makes it more of a craft than engineering, which precludes any guarantee that an alternative prompt would not yield better results. To mitigate this, the three authors applied an assortment of widely accepted prompting patterns from a variety of renowned catalogs, iteratively tweaking the contents of the prompt to assess the effects of small variations, as reported in the results. Instrumentation asymmetry also affects direct cross-model comparisons: we documented exact model identifiers and archived raw outputs to mitigate API drift, but we only explored variations in prompts and

pipelines (sequential chaining, coalescing parallel runs) for LLaMa 3.1, thus the impact of these choices cannot be compared across models.

As the evaluation relies on a single-turn interaction design, internal validity is enhanced by isolating out-of-the-box instruction-following capabilities; however, this precludes insights into iterative human-in-the-loop LLM workflows or dynamic tool use (''copilot'' scenarios).

Regarding construct validity, our conclusions are bounded by three intertwined limitations: 1) a monomethod, monometric, mono-operation design—text-only artifacts exclusively assessed via precision and recall (F2 measure emphasizing omissions)—that omits downstream practical cybersecurity efficacy, stakeholder acceptance, requirement quality, prioritization, or cost; 2) a narrow construct definition that frames success to the coverage of ISO/IEC 27002 technical controls, discouraging out-of-template requirements; and 3) potential metric misalignment, because all errors are weighted equally (regardless e.g., severity or criticality). As we rely on a single human-generated baseline (with its particular granularity), we decouple our analysis from the varying granularity that different LLMs may use when outputting requirements; nevertheless, alternative, equally valid requirement breakdowns may yield different coverage measurements. Other construct threats—label ambiguity between ''excess coverage'' and ''hallucination'', entanglement of the evaluation-frame, and possible reactivity of the judges—were addressed through explicit decision rules and dual reporting against both the manual baseline and the expanded gold standard.

Lastly, the language models studied were highly responsive to the structure and wording of the prompts given; therefore, effectiveness may vary significantly based on how the task is framed—a feature intrinsic to the operation of LLMs. Moreover, our exploration of the effect of prompting strategies offers another independent variable that informs the best practices for automated requirements elicitation.

All in all, our results provide empirical evidence of the capabilities that frontier LLMs can deliver under controlled, single-turn conditions. Our findings should not be construed as a comprehensive benchmark, but rather as a foundation for a follow-up line that incorporates multiple scenarios, natural languages, artifact types, and analyst profiles, together with broader domains, newer model releases, iterative workflows, and further evaluation metrics—directions we outline for future work.

Likewise, as major LLM vendors regularly release new versions whose performance keeps improving (as reflected in public leaderboards and benchmarks), it is plausible that newer versions achieve stronger performance on our specific task as well—although this remains to be empirically verified. Our results should therefore be read as a time-bounded snapshot of the frontier; nonetheless, a natural extension of our work might involve performing a diachronic study that periodically reruns the same pipeline on newer model cohorts to quantify whether and how performance metrics evolve as the LLM frontier shifts.

VII. CONCLUSION

Cybersecurity standards provide structured, reusable best practices; yet, manual instantiation of their control definitions into system-specific requirements remains error-prone and challenging for practitioners without deep expertise. Partial automation of this process, therefore, offers clear benefits: higher requirement quality, earlier reduction of vulnerabilities, and substantial time and cost savings—advantages that make security-by-design more attainable, especially in agile or lean development environments where pace and resource limits often challenge thorough cybersecurity integration. At the same time, the exact magnitude of these savings will depend on how organizations balance the additional coverage provided by LLMs against the effort required to review and curate their outputs, and on the evolving cost–performance profile of LLMs (for example, effective cost per task point).

Our results answer the research questions and show that large language models (LLMs) offer promising—yet still tentative—support for automating cybersecurity requirements generation. Replacing experts entirely would require both near-perfect precision (i.e., effectively no hallucinations) and recall that matches or exceeds the human expert baseline—conditions that are not yet met. However, when discoveries unique to LLMs' outputs are folded into an expanded, pooling-based gold standard, LLMs' recall can surpass human performance. This finding suggests that, in practice, LLMs and humans each tend to surface requirement areas that the other might otherwise overlook. Thus, the most effective role for LLMs in our setting is that of an assisting, complementary tool that enriches the expertise of human engineers rather than as a standalone solution—*more a copilot than an autopilot*—with humans retaining ultimate responsibility for requirement curation and risk acceptance.

Interestingly, our findings show that model provenance was less decisive than expected: open-weights models such as LLaMa match or surpass proprietary counterparts like GPT under our experimental conditions, demonstrating higher coverage with fewer hallucinations.

As expected, effectiveness is highly dependent on input strategy. Our best results came from structured prompt engineering—sequential task decomposition, parallel sampling of multiple outputs, explicit role assignments, chain-of-thought reasoning, and explicit requests for rationales. Moderate hyperparameter settings further helped balance diversity and determinism, yielding more reliable answers; at the same time, our analysis shows that shifts in these settings that improve one dimension (such as recall or diversity) typically degrade another (such as precision or hallucination rate), so gains always come with trade-offs. Finally, and in line with the patterns observed in our run-to-run and hyperparameter analyses, ensemble strategies that combine several runs and models appear more promising than merely adding

TABLE 9. Comparison of related work in the applications of LLMs to requirements engineering and defensive cybersecurity.

Area	Representative prior work	Main focus / task types	Relation / gap with this study
RE – visions, position papers, surveys, and reviews on LLMs	Belzner et al. (2024)[23]; Carleton et al. (2021)[24]; Ozkaya (2023 [25]; Hou et al. (2024)[26]; Norheim & de Weck (2023)[27]; Zadenoori et al. (2025) [28]; Hemmat et al. (2025)[29]; Ferrari (2025)[52]; RE Conference panel on LLMs (2023) [53]	Conceptual discussion of how LLMs / GenAI may support RE throughout the lifecycle. Classify potential LLM roles in RE (elicitation, analysis, traceability, etc.) and map tasks they might support.	Recognize motivation and scenarios for the role of LLMs in RE, identify opportunities and risks. They advocate for LLMs throughout RE tasks, but there is scarce previous work on requirements generation.
RE – automated RE assistance	Ma et al. (SpecGen, 2025) [30]; Tikayat Ray et al. (Aerobert, 2023)[31], [41]; Chen et al. (2023)[32]; Luitel et al. (2024)[38]; Hey et al. (Norbert, 2020)[39]; Luo et al. (Prcbert, 2022)[40]; de Araújo et al. (Re-bert, 2021)[43], etc.	Apply LLMs to generic RE artifacts (user stories, use cases, specs) for quality control, structuring, and traceability.	Focus primarily on classification tasks (based on encoder-only models) for general RE quality and analysis; rather than generation tasks (based on conversational decoder-only models).
Cybersecurity: surveys & reviews on Generative AI	Divakaran & Peddinti; Yao et al. [7]; Zhang et al. (2025)[56]; Shenoy & Mbaziira (2024)[57]; Xu et al. (2024)[60]; Ferrag et al. (2024)[58]; Yigit et al. (2024)[59]	Broad mapping of LLMs/GenAI use in cybersecurity (code analysis, intrusion detection, threat intell., privacy, etc.). Inform how AI might be integrated into security workflows.	Focus on post-development cybersecurity operations, rather than security-by-design for systems under development.
Cybersecurity evaluation suites	Liu (SecQA, 2023)[69]; Tihanyi et al. (CyberMetric)[70]; other LLM security benchmarks cited in SLRs.	Evaluate LLMs on security tasks: Security Operation Centers (SOC) workflows, threat intell., malware summarization, red-teaming, etc.	Assess security-oriented capabilities, but targeting closed-answer questions rather than know-how, and level of SOC/operations rather than standards-based requirements elicitation.
Security requirements / compliance (intersection)	Belzner et al. (2024)[23]; Ronanki et al. (2023)[54]; Bowen (2024)[55].	Use LLMs to relate textual artifacts to trustworthiness or safety frameworks or controls, often at a coarse or classification level.	Move closer to our area, but lack fine-grained instantiation of controls into system-specific requirements.
This study	—	Empirical benchmark of frontier LLMs instantiating ISO/IEC 27002 controls into system-specific cybersecurity requirements for a system under development. Analysis of recall, precision, F-scores; explicit treatment of omissions, hallucinations and discoveries	Bridges RE and cybersecurity. Focuses on standards-based security requirements generation: application to early development artifacts, and analysis of LLM applied cybersecurity know-how with manually curated expert baselines.

more single-model baselines: they can exploit run-to-run and inter-model agreement to prioritize stable requirements and down-weight idiosyncratic hallucinations, thereby increasing coverage while keeping hallucinations in check.

Future research should track the rapid evolution of LLMs, while moving beyond single-turn, text-only scenarios. As LLM capabilities keep improving, the evaluation can be extended to additional families, newer releases, and domain-specialized models. In that sense, our study can be read both as evidence of current potential and as a reusable protocol that can be reapplied longitudinally to future model generations. Further potential extensions may include interactive scenarios where engineers iteratively refine requirements with LLM assistance, and using multimodal descriptions of the target system under development (e.g., architecture diagrams, UI mockups, or even usage videos). It would be useful to explore whether parts of this judgment process can be automated with, for example, semantic-similarity or embedding-based metrics to streamline experimentation, while ensuring that contextual human judgment remains the final arbiter. Moreover, because performance depends heavily on prompt design and hyperparameter configuration, systematic optimization in these areas could unlock further gains.

In any case, confidence in our findings will be strengthened with expanded validation through additional target systems, varied application domains, alternative cybersecurity standards, as well as a broader panel of human experts, which may raise the baseline. A key question is whether current context inputs suffice or whether richer—yet still system-agnostic—guidance mechanisms are needed to ensure robust, realistic generation of cybersecurity requirements. Other key directions for future work to advance towards higher TRLs would also include embedding the proposed hybrid workflow into existing organizational toolchains and validating it in real projects with practicing requirements engineers—addressing effectiveness, usability, governance, and organizational adoption. A complementary direction would investigate how LLMs can help link those requirements to downstream architecture and configuration decisions that implement them.

Ultimately, our findings show that integrating LLM-driven automation into cybersecurity requirements engineering is both promising and complex, highlighting the value that LLMs already add and the fact that careful human stewardship remains essential.

APPENDIX A
SUPPLEMENTARY TABLES

This appendix presents additional tables that complement the main text:

TABLE 10. Detailed hyperparameter values.

	Step	Check		Search		Generate		Coalesce [a]		Format		
	Hyperparameter	T	top-p	T	top-p	T	top-p	T	top-p	T	top-p	Repet. penalty [b]
Model	Configuration											
LLaMa 3.1-405B	L0 (base)	0.6	0.95	0.65	0.95	0.7	0.95	N/A		0.5	0.95	–
	L0a (one-time instructions)	0.6	0.95	0.65	0.95	0.7	0.95	N/A		0.5	0.95	–
	L0b (minimal prompt)	0.6	0.95	0.65	0.95	0.7	0.95	N/A		0.5	0.95	–
	L1 (cooler)	0.6	0.95	0.55	0.85	0.6	0.85	N/A		0.6	0.95	1.03
	L2 (warmer)	0.6	0.95	0.75	0.95	0.8	0.95	N/A		0.6	0.95	1.03
	L3 (4× coalesced)	0.6	0.95	0.65	0.95	0.7	0.95	0.6	0.95	0.6	0.95	–
	L4 (single step) [c]	(←		0.7	0.95		→)	N/A		0.5	0.95	–
	L5 (few-shot)	0.6	0.95	0.75	0.95	0.8	0.95	N/A		0.6	0.95	–
Qwen2-72B	Q0 (base)	0.6	0.95	0.65	0.95	0.7	0.95	N/A		0.5	0.95	N/A
	Q1 (cooler)	0.6	0.95	0.55	0.85	0.6	0.85	N/A		0.5	0.95	N/A
	Q2 (warmer)	0.6	0.95	0.75	0.95	0.8	0.95	N/A		0.5	0.95	N/A
Mixtral-8×22B	M0 (base)	0.6	0.95	0.65	0.95	0.7	0.95	N/A		0.5	0.95	–
	M1 (cooler)	0.6	0.95	0.55	0.85	0.6	0.85	N/A		0.5	0.95	–
	M2 (warmer)	0.6	0.95	0.75	0.95	0.8	0.95	N/A		0.5	0.95	–
GPT-4 Turbo	G0 (base)	0.6	–	0.65	–	0.7	–	N/A		0.5	–	N/A
	G1 (cooler)	0.6	–	0.55	–	0.6	–	N/A		0.5	–	N/A
	G2 (warmer)	0.6	–	0.75	–	0.8	–	N/A		0.5	–	N/A

[a] The coalesce step only appears in the pipeline of configuration L3.
[b] The repetition penalty hyperparameter can only be set in the LLaMa and Mixtral model families.
[c] Configuration L4 executes the first three steps together in a single invocation; therefore, only one value of T and one value of top-p are specified, and they affect all three steps at once.
Empty cells mean that default values were used (no value was explicitly set).
Data extracted from reproducible research package /doc folder. Framework configuration parameters (e.g., Hugging Face and La Plateforme timeout, AWS Bedrock maximum generation length, LangChain verbose) are not listed here.

TABLE 11. LLM leaderboard scores and ranks.

	Open LLM Leaderboard		LM Arena		HELM Capabilities	
Model	Average score	Rank	Rating (score)	Rank	Mean score	Rank
Meta-LLaMa-3.1-405B-Instruct-v1.0	41.7357 [a]	2	1265.532 (bf16)/ 1265.702 (fp8)[b]	11/12	0.618	8
Qwen2-72B-Instruct	42.4863	1	1186.909	58	–	11
Mixtral-8×22B-Instruct-v0.1	33.8857	9	1147.468	83	0.478	19
GPT-4-Turbo-2024-04-09	–	–	1256.745	21	–	–

[a] Open LLM Leaderboard score for Meta-LLaMa-3.1-405B-Instruct-v1.0 is taken from the meta-llama/Llama-3.1-70B-Instruct entry.
[b] LM Arena lists two versions of LlaMa-3.1-405B-Instruct, with different quantization schemes.

- Table 9 contextualizes our study within prior work, grouping related papers by topical area.
- Table 10 details the values of the hyperparameters (temperature, top-p, and other model-specific settings) used in each step and model configuration.
- Table 11 reports the scores and ranks of the LLMs used in this study on several public leaderboards.

APPENDIX B
REPRODUCIBLE RESEARCH

All artifacts required to replicate our evaluation—raw data, prompt libraries, evaluation tables, and result files—are archived on Zenodo under DOI 10.5281/zenodo.15641294 (version 2.0, released on December 9, 2025). The deposit is titled "AI-Augmented Cyber-Security Requirements Generation using LLMs | Reproducible Research Package."

The archive includes, among other resources, the manual baseline of 30 requirements and the gold-standard set of 76 vetted requirements analyzed in Section V.

For a complete directory map, setup instructions, and execution examples, check the repository's README file supplied in the Zenodo record. It should be noted that results are subject to the normal stochastic variance of LLM outputs.

ACKNOWLEDGMENT

AI usage disclosure. During manuscript preparation, the authors employed ChatGPT for editorial purposes—language polishing including grammar correction and style improvement, phrase tightening, content clarification, and improvement of clarity and soundness—in various revisions across the article, and a multi-purpose search engine. ChatGPT and GitHub Copilot were used to streamline code development, always under human oversight. No figures, tables, data analysis, or substantive technical content were generated by AI tools, nor did they contribute to the study design, experimentation, or interpretation of results (besides their role as the subject matter under evaluation in this study).

REFERENCES

[1] OpenAI. (2022). *Introducing ChatGPT*. [Online]. Available: https://openai.com/index/chatgpt/

[2] J. Nielsen. (2023). *AI: First New UI Paradigm in 60 Years*. [Online]. Available: https://www.nngroup.com/articles/ai-paradigm/
[3] T. Lin, Y. Wang, X. Liu, and X. Qiu, "A survey of transformers," *AI Open*, vol. 3, pp. 111–132, Jan. 2022.
[4] R. Bommasani et al., "On the opportunities and risks of foundation models," 2021, *arXiv:2108.07258*.
[5] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. E. Miller, M. Simens, A. Askell, P. Welinder, P. Christiano, J. Leike, and R. Lowe, "Training language models to follow instructions with human feedback," in *Proc. 36th Int. Conf. Neural Inf. Process. Syst. (NIPS)*, 2022, pp. 27730–27744.
[6] W. Xin Zhao et al., "A survey of large language models," 2023, *arXiv:2303.18223*.
[7] Y. Yao, J. Duan, K. Xu, Y. Cai, Z. Sun, and Y. Zhang, "A survey on large language model (LLM) security and privacy: The good, the bad, and the ugly," *High-Confidence Comput.*, vol. 4, no. 2, Jun. 2024, Art. no. 100211.
[8] J. Wei, Y. Tay, R. Bommasani, C. Raffel, B. Zoph, S. Borgeaud, D. Yogatama, M. Bosma, D. Zhou, D. Metzler, E. H. Chi, T. Hashimoto, O. Vinyals, P. Liang, J. Dean, and W. Fedus, "Emergent abilities of large language models," 2022, *arXiv:2206.07682*.
[9] T. J. Wang, A. Roberts, D. Hesslow, T. L. Scao, H. W. Chung, I. Beltagy, J. Launay, and C. Raffel, "What language model architecture and pre-training objective work best for zero-shot generalization?" in *Proc. 39th Int. Conf. Mach. Learn.*, 2022, pp. 22964–22984. [Online]. Available: https://proceedings.mlr.press/v162/wang22u
[10] P. Liu, W. Yuan, J. Fu, Z. Jiang, H. Hayashi, and G. Neubig, "Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing," *ACM Comput. Surv.*, vol. 55, no. 9, pp. 1–35, Sep. 2023.
[11] J. Wei, M. Bosma, V. Y. Zhao, K. Guu, A. W. Yu, B. Lester, N. Du, A. M. Dai, and Q. V. Le, "Finetuned language models are zero-shot learners," in *Proc. 10th Int. Conf. Learn. Represent. (ICLR)*, 2021, pp. 1–46. [Online]. Available: https://openreview.net/forum?id=gEZrGCozdqR
[12] D. Dai, Y. Sun, L. Dong, Y. Hao, S. Ma, Z. Sui, and F. Wei, "Why can GPT learn in-context? Language models secretly perform gradient descent as meta-optimizers," in *Proc. Findings Assoc. Comput. Linguistics, ACL*, 2023, pp. 4005–4019.
[13] T. B. Brown et al., "Language models are few-shot learners," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020, pp. 1877–1901.
[14] P. Kumar, "Large language models (LLMs): Survey, technical frameworks, and future challenges," *Artif. Intell. Rev.*, vol. 57, no. 10, Aug.18, p. 260, 2024.
[15] M. Peeperkorn, T. Kouwenhoven, D. Brown, and A. Jordanous, "Is temperature the creativity parameter of large language models?" in *Proc. 15th Int. Conf. Comput. Creativity*, 2024, pp. 226–235.
[16] L. Li, L. Sleem, N. Gentile, G. Nichil, and R. State, "Exploring the impact of temperature on large language models: Hot or cold?" in *Proc. Int. Neural Netw. Soc. Workshop Deep Learn. Innov. Appl.*, vol. 264, 2025, pp. 242–251.
[17] (2025). *Infrastructure Migration*. [Online]. Available: https://www.llama.com/docs/deployment/infrastructure-migration/
[18] Open Source Initiative. (2024). *The Open Source AI Definition*. [Online]. Available: https://opensource.org/ai/open-source-ai-definition
[19] (2025). *Measuring Thinking Efficiency in Reasoning Models: The Missing Benchmark*. [Online]. Available: https://nousresearch.com/measuring-thinking-efficiency-in-reasoning-models-the-missing-benchmark/
[20] E. Curcio, "Evaluating the lifecycle economics of AI: The levelized cost of artificial intelligence (LCOAI)," *Inf. Syst.*, vol. 136, Feb. 2026, Art. no. 102634.
[21] G. Pan, V. Chodnekar, A. Roy, and H. Wang, "A cost-benefit analysis of on-premise large language model deployment: Breaking even with commercial LLM services," 2025, *arXiv:2509.18101*.
[22] *About—Frontier Model Forum*. Accessed: Jul. 1, 2026. [Online]. Available: https://www.frontiermodelforum.org/about-us/
[23] L. Belzner, T. Gabor, and M. Wirsing, "Large language model assisted software engineering: Prospects, challenges, and a case study," in *Proc. 1st Int. Conf. Bridging Gap Between AI Reality*, 2024, pp. 355–374.
[24] A. Carleton, M. H. Klein, J. E. Robert, E. Harper, R. K Cunningham, D. de Niz, J. T. Foreman, J. B. Goodenough, J. D. Herbsleb, I. Ozkaya, D. Schmidt, and F. Shull. (2021). *Architecting the Future of Software Engineering: A National Agenda for Software Engineering Research & Development*. [Online]. Available: https://www.sei.cmu.edu/library/architecting-the-future-of-software-engineering-a-national-agenda-for-software-engineering-research-development/
[25] I. Ozkaya, "Application of large language models to software engineering tasks: Opportunities, risks, and implications," *IEEE Softw.*, vol. 40, no. 3, pp. 4–8, May 2023.
[26] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, J. Grundy, and H. Wang, "Large language models for software engineering: A systematic literature review," *ACM Trans. Softw. Eng. Methodol.*, vol. 33, no. 8, pp. 1–79, Nov. 2024.
[27] J. J. Norheim and O. L. de Weck, "Large language model applications to space systems engineering," in *Proc. IAF Space Syst. Symp., 74th Int. Astronaut. Congr. (IAC)*, 2023, Paper 80435. [Online]. Available: https://iafastro.directory/iac/archive/browse/IAC-23/D1/4B/80435/
[28] M. A. Zadenoori, J. Dąbrowski, W. Alhoshan, L. Zhao, and A. Ferrari, "Large language models (LLMs) for requirements engineering (RE): A systematic literature review," 2025, *arXiv:2509.11446*.
[29] A. A. Hemmat, M. Sharbaf, S. Kolahdouz-Rahimi, K. Lano, and S. Y. Tehrani, "Research directions for using LLM in software requirement engineering: A systematic review," in *Proc. Frontiers Comp. Sci. (Lausanne)*, vol. 7, Mar. 2025, pp. 1–18. [Online]. Available: https://www.frontiersin.org/journals/computer-science/articles/10.3389/fcomp.2025.1519437/full
[30] L. Ma, S. Liu, Y. Li, X. Xie, and L. Bu, "SpecGen: Automated generation of formal program specifications via large language models," in *Proc. IEEE/ACM 47th Int. Conf. Softw. Eng. (ICSE)*, Apr. 2025, pp. 16–28.
[31] A. Tikayat Ray, B. F. Cole, O. J. Pinon Fischer, A. P. Bhat, R. T. White, and D. N. Mavris, "Agile methodology for the standardization of engineering requirements using large language models," *Systems*, vol. 11, no. 7, p. 352, Jul. 2023.
[32] B. Chen, K. Chen, S. Hassani, Y. Yang, D. Amyot, L. Lessard, G. Mussbacher, M. Sabetzadeh, and D. Varró, "On the use of GPT-4 for creating goal models: An exploratory study," in *Proc. IEEE 31st Int. Requirements Eng. Conf. Workshops (REW)*, Sep. 2023, pp. 262–271.
[33] D. Xie, B. Yoo, N. Jiang, M. Kim, L. Tan, X. Zhang, and J. S. Lee, "How effective are large language models in generating software specifications?" in *Proc. IEEE Int. Conf. Softw. Anal., Evol. Reengineering (SANER)*, Mar. 2025, pp. 1–12.
[34] S. Mandal, A. Chethan, V. Janfaza, S. M. F. Mahmud, T. A. Anderson, J. Turek, J. J. Tithi, and A. Muzahid, "Large language models based automatic synthesis of software specifications," 2023, *arXiv:2304.09181*.
[35] S. Zhang, J. Wang, G. Dong, J. Sun, Y. Zhang, and G. Pu, "Experimenting a new programming practice with LLMs," 2024, *arXiv:2401.01062*.
[36] Y. Wang, L. Shi, M. Li, Q. Wang, and Y. Yang, "A deep context-wise method for coreference detection in natural language requirements," in *Proc. IEEE 28th Int. Requirements Eng. Conf. (RE)*, Aug. 2020, pp. 180–191.
[37] M. Ajagbe and L. Zhao, "Retraining a BERT model for transfer learning in requirements engineering: A preliminary study," in *Proc. IEEE 30th Int. Requirements Eng. Conf. (RE)*, Aug. 2022, pp. 309–315.
[38] D. Luitel, S. Hassani, and M. Sabetzadeh, "Improving requirements completeness: Automated assistance through large language models," *Requirements Eng.*, vol. 29, no. 1, pp. 73–95, Mar. 2024.
[39] T. Hey, J. Keim, A. Koziolek, and W. F. Tichy, "NoRBERT: Transfer learning for requirements classification," in *Proc. IEEE 28th Int. Requirements Eng. Conf. (RE)*, Aug. 2020, pp. 169–179.
[40] X. Luo, Y. Xue, Z. Xing, and J. Sun, "PRCBERT: Prompt learning for requirement classification using BERT-based pretrained language models," in *Proc. 37th IEEE/ACM Int. Conf. Automated Softw. Eng.*, Rochester, MI, USA, Oct. 2022, pp. 1–13.
[41] A. Tikayat Ray, B. F. Cole, O. J. Pinon Fischer, R. T. White, and D. N. Mavris, "AeroBERT-classifier: Classification of aerospace requirements using BERT," *Aerospace*, vol. 10, no. 3, p. 279, Mar. 2023.
[42] A. Sainani, P. R. Anish, V. Joshi, and S. Ghaisas, "Extracting and classifying requirements from software engineering contracts," in *Proc. IEEE 28th Int. Requirements Eng. Conf. (RE)*, Aug. 2020, pp. 147–157.

[43] A. F. de Araújo and R. M. Marcacini, "RE-BERT: Automatic extraction of software requirements from app reviews using BERT language model," in *Proc. 36th Annu. ACM Symp. Appl. Comput.*, Mar. 2021, pp. 1321–1327.

[44] J. Peer, Y. Mordecai, and Y. Reich, "NLP4ReF: Requirements classification and forecasting: From model-based design to large language models," in *Proc. IEEE Aerosp. Conf.*, Mar. 2024, pp. 1–16.

[45] A. El-Hajjami, N. Fafin, and C. Salinesi, "Which AI technique is better to classify requirements? An experiment with SVM, LSTM, and ChatGPT," in *Proc. Workshops, Doctoral Symp., Posters & Tools Track, Educ. Training Track Co-Located 30th Int. Conf. Requirements Eng., Found. Softw. Quality (REFSQ)*, 2024. [Online]. Available: https://ceur-ws.org/Vol-3672/

[46] A. Fantechi, S. Gnesi, L. Passaro, and L. Semini, "Inconsistency detection in natural language requirements using ChatGPT: A preliminary evaluation," in *Proc. IEEE 31st Int. Requirements Eng. Conf. (RE)*, Sep. 2023, pp. 335–340.

[47] A. E. Gärtner and D. Göhlich, "Automated requirement contradiction detection through formal logic and LLMs," *Automated Softw. Eng.*, vol. 31, no. 2, Nov. 2024, Art. no. 49.

[48] S. Lubos, A. Felfernig, T. N. T. Tran, D. Garber, M. El Mansi, S. P. Erdeniz, and V.-M. Le, "Leveraging LLMs for the quality assurance of software requirements," in *Proc. IEEE 32nd Int. Requirements Eng. Conf. (RE)*, Jun. 2024, pp. 389–397.

[49] M. Krishna, B. Gaur, A. Verma, and P. Jalote, "Using LLMs in software requirements specifications: An empirical evaluation," in *Proc. IEEE 32nd Int. Requirements Eng. Conf. (RE)*, Jun. 2024, pp. 475–483.

[50] N. Blasek, K. Eichenmüller, B. Ernst, N. Götz, B. Nast, and K. Sandkuhl, "Large language models in requirements engineering for digital twins," in *Proc. 16th IFIP WG 8.1 Work. Conf. Pract. Enterprise Modeling 13th Enterprise Design Eng. Work. Conf., BES, DTE, FACETE, Tools Demos, Forum, EDEN Doctoral Consortium Co-Located PoEM*, 2023. [Online]. Available: https://ceur-ws.org/Vol-3645/

[51] C. Arora, J. Grundy, and M. Abdelrazek, "Advancing requirements engineering through generative AI: Assessing the role of LLMs," in *Generative AI for Effective Software Development*, Cham, Switzerland: Springer, 2024, pp. 129–148.

[52] A. Ferrari and P. Spoletini, "Formal requirements engineering and large language models: A two-way roadmap," *Inf. Softw. Technol.*, vol. 181, May 2025, Art. no. 107697.

[53] M. Borg, "Requirements engineering and large language models: Insights from a panel," *IEEE Softw.*, vol. 41, no. 2, pp. 6–10, Mar. 2024.

[54] K. Ronanki, C. Berger, and J. Horkoff, "Investigating ChatGPT's potential to assist in requirements elicitation processes," in *Proc. 49th Euromicro Conf. Soft. Eng. Adv. Appl. (SEAA)*, Sep. 2023, pp. 354–361.

[55] M. B. Bowen, "Leveraging latent textual topology for standard identification in engineering design," M.S. thesis, College of Eng., Univ. Georgia, Athens, GA, USA, 2024.

[56] J. Zhang, H. Bu, H. Wen, Y. Liu, H. Fei, R. Xi, L. Li, Y. Yang, H. Zhu, and D. Meng, "When LLMs meet cybersecurity: A systematic literature review," *Cybersecurity*, vol. 8, no. 1, Feb. 2025, Art. no. 55.

[57] N. Shenoy and A. V. Mbaziira, "An extended review: LLM prompt engineering in cyber defense," in *Proc. Int. Conf. Electr., Comput. Energy Technol. (ICECET)*, Jul. 2024, pp. 1–6.

[58] M. Amine Ferrag, F. Alwahedi, A. Battah, B. Cherif, A. Mechri, N. Tihanyi, T. Bisztray, and M. Debbah, "Generative AI in cybersecurity: A comprehensive review of LLM applications and vulnerabilities," 2024, *arXiv:2405.12750*.

[59] Y. Yigit, W. J. Buchanan, M. G. Tehrani, and L. Maglaras, "Review of generative AI methods in cybersecurity," 2024, *arXiv:2403.08701*.

[60] H. Xu, S. Wang, N. Li, K. Wang, Y. Zhao, K. Chen, T. Yu, Y. Liu, and H. Wang, "Large language models for cyber security: A systematic literature review," *ACM Trans. Softw. Eng. Methodol.*, Sep. 2025. [Online]. Available: https://dl.acm.org/doi/10.1145/3769676

[61] M. T. Alam, D. Bhusal, L. Nguyen, and N. Rastogi, "CTIBench: A benchmark for evaluating LLMs in cyber threat intelligence," in *Proc. Adv. Neural Inf. Process. Syst.*, Jun. 2024, pp. 50805–50825.

[62] G. Siracusano, D. Sanvito, R. Gonzalez, M. Srinivasan, S. Kamatchi, W. Takahashi, M. Kawakita, T. Kakumaru, and R. Bifulco, "Time for aCTIon: Automated analysis of cyber threat intelligence in the wild," 2023, *arXiv:2307.10214*.

[63] N. Roch, H. Sievers, L. Schöni, and V. Zimmermann, "Navigating autonomy: Unveiling security experts' perspectives on augmented intelligence in cybersecurity," in *Proc. 20th Symp. Usable Privacy Secur. (SOUPS)*, 2024, pp. 41–60.

[64] W. Alhoshan, A. Ferrari, and L. Zhao, "Zero-shot learning for requirements classification: An exploratory study," *Inf. Softw. Technol.*, vol. 159, Jul. 2023, Art. no. 107202.

[65] J. von der Mosel, A. Trautsch, and S. Herbold, "On the validity of pre-trained transformers for natural language processing in the software engineering domain," *IEEE Trans. Softw. Eng.*, vol. 49, no. 4, pp. 1487–1507, Apr. 2023.

[66] Y. Chang, X. Wang, J. Wang, Y. Wu, L. Yang, K. Zhu, H. Chen, X. Yi, C. Wang, Y. Wang, W. Ye, Y. Zhang, Y. Chang, P. S. Yu, Q. Yang, and X. Xie, "A survey on evaluation of large language models," *ACM Trans. Intell. Syst. Technol.*, vol. 15, no. 3, pp. 1–45, Mar. 2024.

[67] A. Fan, B. Gokkaya, M. Harman, M. Lyubarskiy, S. Sengupta, S. Yoo, and J. M. Zhang, "Large language models for software engineering: Survey and open problems," in *Proc. IEEE/ACM Int. Conf. Softw. Eng., Future Softw. Eng. (ICSE-FoSE)*, May 2023, pp. 31–53.

[68] K. Ronanki, B. Cabrero-Daniel, J. Horkoff, and C. Berger, "Requirements engineering using generative AI: Prompts and prompting patterns," in *Generative AI for Effective Software Development*. Cham, Switzerland: Springer, 2024, pp. 109–127.

[69] Z. Liu, "SecQA: A concise question-answering dataset for evaluating large language models in computer security," 2023, *arXiv:2312.15838*.

[70] N. Tihanyi, M. A. Ferrag, R. Jain, T. Bisztray, and M. Debbah, "CyberMetric: A benchmark dataset based on retrieval-augmented generation for evaluating LLMs in cybersecurity knowledge," in *Proc. IEEE Int. Conf. Cyber Secur. Resilience (CSR)*, Sep. 2024, pp. 296–302.

[71] J. Gennari, S.-H. Lau, S. J. Perl, J. Parish, and G. Sastry. (2024). *Considerations for Evaluating Large Language Models for Cybersecurity Tasks*. [Online]. Available: https://www.sei.cmu.edu/library/considerations-for-evaluating-large-language-models-for-cybersecurity-tasks/

[72] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, L. Zi, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, "Judging LLM-as-a-judge with MT-bench and chatbot arena," in *Proc. 37th Int. Conf. Neural Inf. Process. Syst. (NIPS)*, Jun. 2023, pp. 46595–46623.

[73] C. Fourrier, N. Habib, A. Lozovskaya, K. Szafer, and T. Wolf. (2024). *Open LLM Leaderboard V2*. [Online]. Available: https://huggingface.co/spaces/open-llm-leaderboard/open_llm_leaderboard

[74] H. Li, Q. Dong, J. Chen, H. Su, Y. Zhou, Q. Ai, Z. Ye, and Y. Liu, "LLMs-as-judges: A comprehensive survey on LLM-based evaluation methods," 2024, *arXiv:2412.05579*.

[75] T. Li, W.-L. Chiang, E. Frick, L. Dunlap, T. Wu, B. Zhu, J. E. Gonzalez, and I. Stoica, "From crowdsourced data to high-quality benchmarks: Arena-hard and benchbuilder pipeline," in *Proc. 42nd Int. Conf. Mach. Learn. ICML*, Jun. 2024, pp. 34209–34231. [Online]. Available: https://openreview.net/forum?id=KfTf9vFvSn

[76] W.-L. Chiang, L. Zheng, Y. Sheng, A. N. Angelopoulos, T. Li, D. Li, B. Zhu, H. Zhang, M. I. Jordan, J. E. Gonzalez, and I. Stoica. "Chatbot arena: An open platform for evaluating LLMs by human preference," in *Proc. 41st Int. Conf. Mach. Learn. (ICML)*, Mar. 2024, pp. 8359–8388.

[77] P. Liang et al., "Holistic evaluation of language models," *Trans. Mach. Learn. Res.*, Aug. 2023. [Online]. Available: https://openreview.net/pdf?id=iO4LZibEqW

[78] F. Massacci, J. Mylopoulos, and N. Zannone, "Security requirements engineering: The SI modeling language and the secure tropos methodology," in *Advances in Intelligent Information Systems* (Studies in Computational Intelligence), vol. 265. Berlin, Germany: Springer, 2010, pp. 147–174.

[79] A. van Lamsweerde, "Elaborating security requirements by construction of intentional anti-models," in *Proc. 26th Int. Conf. Softw. Eng.*, pp. 148–157.

[80] H. Mouratidis, P. Giorgini, and G. A. Manson, "An ontology for modelling security: The tropos approach," in *Proc. Knowl.-Based Intell. Inf. Eng. Syst. (KES)*, 2003, pp. 1387–1394.

[81] Common Criteria Recognition Arrangement (CCRA). (2022). *Common Criteria for Information Technology Security Evaluation*. [Online]. Available: https://www.commoncriteriaportal.org/cc/index.cfm

[82] *Security and Privacy Controls for Information Systems and Organizations*, Standard NIST SP 800–53, 2020, doi: 10.6028/NIST.SP.800-53r5.

[83] *Control Baselines for Information Systems and Organizations*, Standard NIST SP 800-53B, 2020, doi: 10.6028/NIST.SP.800-53B.

[84] *Risk Management Framework for Information Systems and Organizations: A System Life Cycle Approach for Security and Privacy*, Standard NIST SP 800-37, 2018, doi: 10.6028/NIST.SP.800-37r2.
[85] S. Wang, S. Xu, and A. Horowitz. (2024). *16 Changes to the Way Enterprises Are Building and Buying Generative AI*. [Online]. Available: https://a16z.com/generative-ai-enterprise-2024/
[86] OpenRouter. *LLM Rankings*. [Online]. Available: https://openrouter.ai/rankings,https://web.archive.org/web/20250716223421/,https://openrouter.ai/rankings
[87] J. White, Q. Fu, S. Hays, M. Sandborn, C. Olea, H. Gilbert, A. Elnashar, J. Spencer-Smith, and D. C. Schmidt, "A prompt pattern catalog to enhance prompt engineering with ChatGPT," in *Proc. 30th Conf. Pattern Lang. Programs*, Monticello, IL, USA, 2023, pp. 1–31.
[88] S. Schulhoff et al., "The prompt report: A systematic survey of prompt engineering techniques," 2024, *arXiv:2406.06608*.
[89] P. Sahoo, A. K. Singh, S. Saha, V. Jain, S. Mondal, and A. Chadha, "A systematic survey of prompt engineering in large language models: Techniques and applications," 2024, *arXiv:2402.07927*.
[90] B. Chen, Z. Zhang, N. Langrené, and S. Zhu, "Unleashing the potential of prompt engineering for large language models," *Patterns*, vol. 6, no. 6, Jun. 2025, Art. no. 101260.
[91] *Prompt Engineering Guide*. Accessed: Feb. 7, 2025. [Online]. Available: https://www.promptingguide.ai/
[92] G. Marvin, N. Hellen, D. Jjingo, and J. Nakatumba-Nabende, "Prompt engineering in large language models," in *Proc. Data Intell. Cognit. Inform. (ICDICI)*, 2024, pp. 387–402.
[93] K. Sparck-Jones, and C. J. Van Rijshergen. (1975). *Report on the Need for and Provision of an Ideal Information Retrieval Test Collection*. [Online]. Available: https://sigir.org/files/museum/pub-14/pub_14.pdf
[94] E. M. Voorhees, "The philosophy of information retrieval evaluation," in *Proc. Eval. Cross-Lang. Inf. Retr. Syst. 2nd Workshop Cross-Lang. Eval. Forum*, 2002, pp. 355–370.
[95] Arena Intelligence. (2024). *Text Arena | LMArena*. [Online]. Available: https://lmarena.ai/leaderboard/text,https://huggingface.co/spaces/lmarena-ai/lmarena-leaderboard/blob/main/elo_results_20240915.pkl
[96] HuggingFace. (2025). *Open LLM Leaderboard Archived*. [Online]. Available: https://huggingface.co/spaces/open-llm-leaderboard/open_llm_leaderboard#/,https://huggingface.co/datasets/open-llm-leaderboard/contents/tree/2f9a8f919661132e2f904c3ac59810facd94c4d7
[97] *Stanford Institute for Human-Centered Artificial Intelligence (HAI). (20 March). Capabilities-Holistic Evaluation of Language Models (HELM)*, Center for Research on Foundation Models (CRFM), Stanford, CA, USA, Mar. 2025. [Online]. Available: https://crfm.stanford.edu/helm/capabilities/v1.0.0/
[98] N. R. Mead, E. D. Hough, and T. H. R. Stehney. (2005). *Security Quality Requirements Engineering (SQUARE) Methodology*. [Online]. Available: https://www.sei.cmu.edu/library/security-quality-requirements-engineering-technical-report/
[99] OWASP Foundation. (2022). *Software Assurance Maturity Model*. [Online]. Available: https://owaspsamm.org/model/
[100] (2005). *The CLASP Application Security Process*. [Online]. Available: https://cwe.mitre.org/documents/sources/TheCLASPApplicationSecurityProcess.pdf



JUAN C. YELMO received the M.S. and Ph.D. degrees in telecommunications engineering from Universidad Politécnica de Madrid, Madrid, Spain, in 1991 and 1996, respectively.

He is currently an Associate Professor with Universidad Politécnica de Madrid, where he has more than 25 years of experience in software and internet services engineering. He has participated in numerous national and international research projects and has been involved in technology transfer and innovation activities, including research contracts with international companies, standardization efforts, open innovation initiatives, and consulting for university spin-offs. His current research interests include cybersecurity and AI-augmented software engineering.



YOD-SAMUEL MARTÍN (Member, IEEE) received the M.Sc. degree in telecommunications engineering from Universidad Politécnica de Madrid (UPM), in 2004, and the M.Phil. and Ph.D. degrees, in 2006 and 2022, respectively.

He has held different research and lecturing positions at UPM: the Center for Open Middleware (COM), the Information Processing and Telecommunications Center (IPTC), and the Departamento de Ingeniería de Sistemas Telemáticos (DIT), where he is currently an Assistant Professor. His research interests include the management of different categories of extra-functional requirements (cybersecurity, privacy, accessibility, usability, and internationalization) throughout the systems development life cycle.



SANTIAGO PEREZ-ACUNA received the B.Sc. degree in computer engineering and business administration from the University of Vigo, Spain, and the M.Sc. degree in network and telematics services engineering from Universidad Politécnica de Madrid.

He is currently a Predoctoral Researcher with the Department of Telematics Engineering, Universidad Politécnica de Madrid (DIT-UPM). His research focuses on the application of artificial intelligence to cybersecurity in industrial software systems. His main research interests include AI-assisted requirements engineering, large language models, and the development of secure and trustworthy software in industry 4.0 contexts.